

Research article

EDITORS: Energy-aware Dynamic Task Offloading using Deep Reinforcement Transfer Learning in SDN-enabled Edge Nodes

Thar Baker^{a,*}, Zaher Al Aghbari^b, Ahmed M. Khedr^b, Naveed Ahmed^b, Shini Girija^b

^a School of Architecture, Technology and Engineering, University of Brighton, Brighton BN2 4GJ, UK

^b Department of Computer Science, University of Sharjah, Sharjah, 27272, United Arab Emirates

ARTICLE INFO

Keywords:

Dynamic task offloading
Trust
Energy efficiency
Deep Reinforcement Transfer Learning
Software defined networks

ABSTRACT

In mobile edge computing systems, a task offloading approach should balance efficiency, adaptability, trust management, and reliability. This approach aims to maximise resource utilisation, improve user experience, and satisfy application-specific requirements while taking into account the dynamic and limited resource nature of edge environments. Additionally, while offloading tasks, these systems are vulnerable to several attacks and privacy breaches, necessitating edge node trust evaluation. However, not all of these necessary features are present in the offloading methods currently used. This research proposes 'EDITORS' (energy-aware dynamic task offloading method utilising Deep Reinforcement Transfer Learning (DRTL) in Software-Defined Network (SDN) enabled edge computing environments), a novel approach aimed at addressing the multifaceted issues associated with offloading in mobile edge computing systems. The primary goal of EDITORS is to design a task offloading system that incorporates trusted edge nodes while prioritising energy efficiency, timeliness, reliability, adaptability, and outperforming existing task offloading methods in terms of the quality of the task offloading plan. This method uses of DRTL agents at edge nodes, which communicate with the SDN controller to learn the most appropriate offloading choices based on network conditions and resource availability. Extensive simulations (six) are conducted which show that the EDITORS significantly increases energy efficiency while preserving low-latency task completion compared to five existing offloading methods (DDLO, DROO, DMRO, DRL without TL and SDN and DRL with SDN). EDITORS includes trust evaluation, trusted edge device prediction using LSTM, and adaptation of newly added devices through transfer learning, unlike other task offloading methods that just concentrate on task offloading.

1. Introduction

Mobile edge computing (MEC) has emerged as a key technology for enhancing the performance of time-sensitive and computation-intensive Internet of Things (IoT) applications like smart cities, smart grids, wearables, and industrial automation, which demand substantial energy, memory, and computational resources [1,2]. While IoT devices have seen power improvements, their batteries, CPUs, and memory still lack the capacity for running resource-intensive applications. MEC tackles this challenge by enabling computational task distribution to near IoT devices, offering an advanced solution that reduces execution time, minimises latency, improves reliability, and enhances energy efficiency [3]. However, edge nodes have a limited amount of processing resources, such as CPU/GPU power and memory, due to their restrictions on size, weight, and energy [4].

* Corresponding author.

E-mail addresses: T.Shamsa@brighton.ac.uk (T. Baker), zaher@sharjah.ac.ae (Z. Al Aghbari), akhedr@sharjah.ac.ae (A.M. Khedr), nahmed@sharjah.ac.ae (N. Ahmed), p20230011@dubai.bits-pilani.ac.in (S. Girija).

<https://doi.org/10.1016/j.iot.2024.101118>

Received 9 September 2023; Received in revised form 28 January 2024; Accepted 7 February 2024

Available online 10 February 2024

2542-6605/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Dynamic partial task offloading can be used as an alternate way to deal with the difficulty of performing a large-scale task on a single node. This method significantly increases overall efficiency by processing different parts of each task in parallel across several edge nodes [5,6]. Consider a person re-identification [7] application that uses real-time video analysis to locate a missing person as an example. These systems are built to handle dynamic and intense workloads and are workload-aware. The substantial video processing workload significantly affects the performance of the end device. Many of the earlier models of edge nodes [7–10] have been over simplified when handling computation-intensive tasks without actually characterising the impact of workload. Even if the end device's computational capabilities are adequate, fully local video processing consumes a lot of energy. Even if moving entire video footage to a close-by single edge node saves energy, response time can still increase significantly. Offloading video streams to many edge nodes for parallel processing is a more efficient solution that greatly reduces response time compared to full local execution.

Although dynamic partial task offloading can address edge nodes' drawbacks, there are still issues of selecting trusted devices, minimising energy consumption, and minimising latency that need to be evaluated [11,12]. It is essential to have a distributed intelligent approach at the edge that manages distributed computing, networking, collaboration, and storage resources. Making the most appropriate offloading choice at the edge, however, is challenging due to the heterogeneity of devices (i.e., power and capacity). As a result, dynamic partial task offloading in edge nodes presents a wide range of issues including the need to make decisions about, (i) identifying the types of tasks suitable for offloading; (ii) the ideal number of tasks to be offloaded; (iii) how to select trusted edge nodes to handle task allocation and resource planning; (iv) mapping of incoming tasks to potential edge nodes according to their specific resource capacities; and (v) how to carry out collaborative processing of sub-tasks among edge nodes.

The procedure for recognising tasks suitable for offloading entails an evaluation of the inherent characteristics and resource demands of these tasks [13]. Suitable tasks for offloading are generally those that can take advantage of the computational capabilities offered by edge nodes. This may include computationally intensive tasks, tasks with high data processing requirements, or tasks that require low-latency responses. Task profiling and classification techniques can identify such tasks [14]. Selecting the ideal number of tasks to offload is essential for managing resources [15]. The right amount of task offloading is important to avoid underutilising the edge resources that are available or creating network congestion and resource contention from offloading too few tasks. This decision-making process requires considering several factors, including task priority, device capabilities, and network conditions [16]. Choosing trustworthy edge nodes involves evaluating their security and reliability [17]. Edge node trustworthiness can be assessed using a trust management technique. Trusted nodes can be chosen based on criteria such as device reputation, past performance, and security measures [18]. This is essential to guarantee that the tasks that have been offloaded are completed safely and efficiently. Understanding the unique resource capacities and capabilities of each edge device is necessary for the mapping of incoming tasks to edge nodes [19]. It entails balancing the demands of the task – such as memory, energy consumption, and communication bandwidth – with the resources of edge nodes. To optimise resource utilisation and reduce execution time, this task can benefit from the application of resource-aware scheduling methods and optimisation techniques [20].

The offloading process is typically optimised by breaking tasks down into subtasks [21]. Subtask division can support resource utilisation and load balancing [22]. Subtasks that are crucial or time-sensitive, for instance, might be carried out locally, whereas non-critical or computationally demanding subtasks could be offloaded to the edge or cloud [23]. Moreover, subtask division allows for greater flexibility in the management of resource limitations, including memory, bandwidth, latency and energy [21]. It makes it possible to allocate resources more precisely and optimise the location of execution.

Software-Defined Network (SDN) creates intelligent edge services that are dynamically re-configurable by separating upper-layer IoT applications from the foundational physical edge resources [24,25]. Tasks from various end devices connected to intelligent edge nodes are collected and exchanged to train models. These models are subsequently utilised for computation and predictions [26]. This strategy offers logical and centralised control over distributed edge nodes and IoT devices. However, there have not been many studies on applying SDN for the subtask co-processing through dynamic partial offloading among edge nodes in the IoT environment. In order to achieve this, this paper suggests a distributed SDN controller edge-based architecture that divides the control plane from the data plane and enables the controller to gather data from edge nodes and perceive network status from a global perspective. Edge nodes can share their data with other system nodes via a distributed SDN controller, allowing the controller to choose whether the present task should be carried out locally or dispersed around several edge servers for collaborative processing by checking the status of resources such as energy consumption, execution time and bandwidth. This enables numerous edge nodes to work together for on-demand computing resource distribution in a highly automated and intelligent way.

Another challenge in offloading tasks to edge nodes is selecting trustworthy nodes to participate in the process. The term “trust in computation offloading” describes the reliance and confidence an end user device places in the ability of an edge node to precisely and reliably carry out the anticipated computational tasks as stated by the device [27]. Existing methods suggest leveraging a node's resource attributes to make sure it has enough resources to complete the task [28,29]. However, establishing trust among edge nodes and end user devices should be included in the process included in task offloading. This trust mechanism aims to identify edge nodes that excessively or inadequately utilise their resources during task execution [30]. This study suggests an intelligent edge collaboration-based feedback trust management technique to choose the most reliable and trustworthy devices in terms of energy consumption and execution time. Feedback mechanisms calculate trust scores for each edge node by evaluating the devices' trustworthiness based on their prior interactions and network behaviour. For instance, an edge device that consistently performs well might be given a high trust score and utilised for more difficult offloading tasks. In contrast, a device that performs poorly might be given a low trust score and only used for less significant tasks.

Determining the most appropriate device, allocating resources, and maintaining data security are all difficult choices that must be made during the task offloading process between edge devices. Trusted devices provide a reliable method for offloading work, but

performance or resource usage may not be optimised [31,32]. This paper proposes the utilisation of a DRTL-based algorithm within an SDN controller for task offloading. By incorporating rewards and penalties, this approach trains trusted devices to define the most appropriate offloading policy among edge nodes. The primary goal is to design a task offloading system that incorporates trusted edge nodes while prioritising energy efficiency, timeliness, reliability, adaptability, and outperforming existing task offloading methods in terms of the quality of the task offloading plan. To adapt to the environment and choose the most appropriate offloading strategy, newly linked devices only need a minimal amount of training using DRTL. In this regard, the DRTL's performance with SDN is assessed and contrasts it to the performance of the current deep learning-based offloading techniques without SDN. The results of the experiments demonstrate that this strategy can efficiently increase task offloading efficiency while lowering energy consumption and execution time.

In this paper, we investigate a person re-identification application on the MEC platform with several IoT devices and edge nodes. We carefully examine the task offloading issue in this scenario and make the following main contributions:

- A distributed SDN controller architecture that facilitates collaboration across several edge nodes is proposed for the deployment of on-demand computing resources. In a distributed architecture, edge nodes can employ distributed SDN controllers to exchange data among themselves, empowering the controller to make highly accurate evaluations of task execution. The controller determines whether the current task should be executed locally or distributed among several edge nodes for collaborative work by taking into account factors like energy consumption, execution time, and latency. This proposed architecture offers benefits including enhanced resource efficiency, intelligent resource allocation, distributed control, and optimised task execution, improving overall performance in task offloading. Section 3 presents the proposed distributed SDN controller architecture for dynamic partial workload offloading.
- A machine-learning based feedback trust management technique using LSTM is presented to choose the most dependable and trustworthy devices based on energy consumption, latency and memory usage. The trust score is determined as a weighted sum of factors, such as energy consumption, latency and memory usage, which affect the device's overall trustworthiness. The use of a feedback mechanism for trust management in MEC systems increases service quality, lowers the possibility of security breaches, and boosts task offloading's performance, reliability, and efficiency. Section 4 presents the proposed feedback-based trust management using LSTM.
- A DRTL algorithm in distributed SDN architecture-EDITORS is proposed that can efficiently lower the cost (in terms of latency and energy consumption of the task computation) of locating the most appropriate edge node, speed up task completion, and cut down on energy consumption and latency of task offloading and execution. This DRTL is based on intelligent agents that update their assessments based on their interactions with the environment, optimising consumable energy usage while minimising latency and training time. Section 5 presents the proposed DRTL-based task offloading algorithm with a distributed SDN, EDITORS approach.
- Six comprehensive experiments were carried out to validate the effectiveness of the DRTL method with SDN. The research comprised benchmarking against three cutting-edge task offloading methods without SDN, and the results showed a significant decrease in task completion time, latency, and energy consumption. Section 7 discusses the results of various experiments conducted to validate the EDITORS method.

The paper is organised as follows: Section 2 discusses related work. Section 3 presents the software-defined edge-computing architecture. Section 4 presents the proposed feedback-based trust management using LSTM. Section 5 depicts the EDITORS, the proposed DRTL-based task offloading method with distributed SDN controller. Section 6 presents the details of performance evaluation. We conducted extensive experiments to evaluate the algorithm and present the evaluation results in Section 7. Section 8 concludes the paper.

2. Related works

The authors examine previous research concerning SDN-enabled task allocation, feedback-based trust computing, and DRTL-based task offloading in this section.

2.1. SDN-enabled task allocation

IoT applications can benefit from centralised control, an abstracted network view, and adaptable rule-based forwarding when they employ a software-defined network (SDN) architecture [33]. With the help of SDN, the controller can gather network data from multiple wireless devices and base task offloading decisions on the overall network view. SDN also offers flexibility for coordinating and managing edge-based IoT environments and simplifies network management. Due to these advantages, SDN stands out as an attractive option for resolving task offloading problems in dynamic IoT settings [34]. Lia et al. [35] developed a machine learning-based task placement system that works with a centralised SDN controller to carefully plan the placement of latency-constrained computing tasks in MEC systems. They receive off-line training based on the outcomes of an optimisation problem that aims to minimise the use of edge network resources while honouring task-specific latency limitations. However, utilising a centralised controller can face scalability and latency challenges in networks that span multiple domains or operate at a large scale. Kiran

et al. [34] introduced a novel method for task offloading and resource distribution in MEC environments by merging SDN and MEC known as the software-defined edge cloudlet paradigm. The network state is monitored by a central logical controller, which also gathers data from user equipment task data (such as computation requirements) and distributed edge server data (such as available memory and CPU load). Based on this data, the controller advises devices whether to finish the task locally or offload it to an edge or distant cloud for additional processing. However, in such a network configuration, a single centralised SDN controller can run into problems like excessive control latency and bandwidth restrictions. The usage of a distributed SDN controller is clearly the most appropriate option for task offloading in cases involving enormous task offloading after these constraints of a centralised SDN controller have been examined.

Limited research has been conducted on the implementation of distributed SDN controllers among edge nodes for dynamic task offloading [4,12]. Zhang et al. [4] introduced a SDN edge-computing architecture that provides the controller with a comprehensive view of the computational resources available at the network edge. However, this methodology places numerous constraints and presumptions on the networks, many of which have little relation to real MEC systems. Sellami et al. [12] addressed the problem of scheduling energy-conscious and low-latency computing tasks in a Software-Defined Fog-IoT Network. It formulates the issue using Deep Q-Learning and introduces a Deep Reinforcement Learning (DRL) method for dynamic task scheduling and assignment in SDN-enabled edge networks with the goal of maximising energy efficiency and lowering latency. However, collaboration among edge nodes is not explored.

In light of the aforementioned challenges, the proposed distributed SDN controller architecture enables intelligent edge nodes to work together to allocate subtasks in the most efficient way possible by analysing state data in terms of energy consumption, latency, and trust score. The suggested architecture thus meets the three key criteria for dynamic task offloading: reliability, effectiveness, and performance.

2.2. Feedback-based trust computing

Mujawar et al. [36] introduced a method to evaluate the trustworthiness of a cloud service provider by analysing its behaviour and user feedback. Multiple service-level agreement parameters are employed to record feedback and establish the trustworthiness of the service provider based on user feedback. Cumulative trust, which is derived from behavioural trust and feedback trust, is used to assess the reliability of cloud service providers. However, only user feedback is taken into account for calculating the trust score. Liu et al. [37] proposed a trustworthy collaboration service ecosystem for a collaborative cloud service environment. The ecosystem incorporates cloud service brokerages (CSBs) as intermediaries and introduces a bi-feedback mechanism among cloud consumers, CSBs, and cloud service providers. While the model provides a trust-aware collaboration service and leverages social relationships for feedback, the use of a broker to collect feedback introduces bottlenecks and increased computational overhead, which should be addressed. Rjoub et al. [30] developed a technique to establish trust among edge nodes and IoT devices since they contend that it should be a factor in decision-making for offloading. The trust mechanism spotted the devices that used their resources wastefully or excessively during local training. The statistical modified Z-score approach is used to calculate the trust score. Since this method does not consider the feedback from different entities in the system, the accuracy of the trust score is less.

Under the scope of the aforementioned challenges, a machine learning-based feedback trust management technique is proposed for selecting trusted edge nodes for task offloading within the context of a SDN controller. The technique involves calculating a trust score based on feedback received from various system entities, including cloud servers, other edge nodes, and IoT devices. The distributed SDN controller helps to collect these feedback from network. Furthermore, a machine learning model can be employed to predict the trust score using historical feedback data obtained from the SDN controller. This approach aims to resolve the challenges of trust management and resource allocation in a dynamic MEC system.

2.3. DRL-based task offloading in SDN controller architecture

Traditional reinforcement learning (RL) algorithms, such as Deep Q-Network (DQN) [38] and Q-Learning [39], have been extensively used for task offloading in edge clouds due to their compatibility with the nature of the issue. These traditional RL algorithms, however, might not deliver appropriate outcomes as system complexity rises [31]. The handling of the dynamic environment in the context of task offloading across edge clouds has been proposed using DRL-based algorithms. Deep neural networks (DNN) are incorporated into these algorithms, which show better capabilities for handling huge state and action spaces. Multi-agent DRL, which treats each device as an agent exploring the environment, has been used in many studies [5,12,35]. These strategies combine inquiry with shared experiences-based policy training. The variety of devices still needs study, even if several recent efforts have addressed adaptation to dynamic contexts and included collaborative training or global state concerns.

Therefore, by integrating DRL and Transfer Learning (TL) into the DRTL framework [40], the objective is to diminish the disparity between device heterogeneity and the significant expenses linked to policy training, all while achieving an optimal task offloading solution with minimal energy consumption and latency. The integration of SDN with the DRTL-based algorithm helps enhance network management, resource allocation, and adaptation to the changing environment in a more efficient and scalable manner.

Numerous offloading strategies have been put out in the literature to reduce total cost in terms of energy consumption and task execution latency. Table 1 displays an overview of the current contributions. While aiming to achieve diverse Quality of Service (QoS) objectives, such as latency reduction and energy efficiency, most prior solutions primarily focus on optimal scheduling and computation offloading methods as distinct entities [38,41]. In [38], Huang et al. proposed the distributed deep learning-based offloading technique, DDLO, for MEC networks to reduce the overall system utility, which includes both the total energy consumption

Table 1
Comparison with existing works.

Existing works	Computation offloading	Task scheduling	Device selection	Trust management	SDN	TL
[38]	Yes	Yes	No	No	No	No
[41]	Yes	Yes	No	No	No	No
[42]	Yes	Yes	Yes	No	No	No
[43]	Yes	Yes	Yes	No	No	No
[12]	Yes	Yes	Yes	No	Yes	No
[4]	Yes	Yes	Yes	No	Yes	No
[44]	Yes	Yes	Yes	No	Yes	No
[45]	Yes	Yes	Yes	No	No	No
[46]	Yes	Yes	Yes	No	Yes	No
[47]	Yes	Yes	Yes	No	No	No
EDITORS	Yes	Yes	Yes	Yes	Yes	Yes

and the task completion latency. The approach uses of many deep neural networks and produces nearly ideal results without the need for manually labelled input. The biggest drawback is the lack of a suitable device selection strategy for task offloading. Using a deep neural network as a scalable solution, Huang et al. [41] suggested the DRL-based Online Offloading (DROO) framework, which learns the binary offloading decisions from the experience. DROO can solve combinatorial optimisation problems, significantly lowering computing complexity, especially for large-scale networks. However, the mobility of wireless devices would make DROO difficult to converge, which is a major challenge. In [42], Elgendy et al. proposed an effective computation offloading and task caching model for a multi-user, multi-task MEC system. For this, algorithms based on Q-learning and Deep-Q-Networks have been presented, which can effectively handle the offloading problem and arrive at the almost ideal answer. The modelling and experimental results demonstrated that offloading and caching tasks at the edge server can greatly reduce energy and time consumption. The biggest drawback is that it cannot be used in dynamic environments. In [43], Nguyen et al. adopted a DRL algorithm, which enables the client to learn and discover the most appropriate choices without prior knowledge of network settings. The experiment findings show that the suggested algorithm performs better in terms of computation latency and incentive cost than baseline and standard Q-learning techniques. The biggest drawback is the absence of task division, which can lower overall costs and latency. In [12], Sellami et al. demonstrated the potential of creating task assignment and scheduling systems for SDN-enabled IoT networks. The authors came up with a task assignment and scheduling issue that maximises energy efficiency while reducing network latency. The biggest drawback is the lengthy training period and the uncertain reliability of some offloading fog nodes. In [4], Zhang et al. proposed a reinforcement-learning-based software-defined edge task allocation algorithm (RL-SDETA) is put forth in the software-defined IoT, allowing controllers to assign computational tasks to the edge servers that are best suited to carry them out based on the information they have learned about the entire network. The key drawbacks are that training takes a lot of time and that certain offloading fog nodes are not entirely reliable. In [44], Zhang et al. presented an SDN-assisted MEC system architecture tailored for vehicular networks, where the SDN controller efficiently gathers and interprets vehicle and MEC information to enable optimal radio access interface selection for data transmission. The study's main drawback is how little attention it pays to how to handle the problem of user service disruptions in high-speed vehicle networks, which could pose serious problems in real-world deployment scenarios. In [45], Zhao et al. introduced the Digital Twin-Assisted Intelligent Partial Offloading scheme (IGNITE) as a promising approach to efficiently manage complex service requests in vehicular edge computing systems. IGNITE optimises both computational latency and service price, outperforming previous task offloading strategies through vehicle clustering, DRL-based offloading, and a feedback mechanism. Nevertheless, this method's current emphasis on computing efficiency and service cost optimisation obscures possible security and privacy issues in the vehicular network, which are crucial for real-world deployment. In [46], Zhao et al. introduced the UAV-assisted Vehicular computation Cost Optimization (UVCO) algorithm, which utilises a UAV to aid legacy vehicles in executing time-sensitive and resource-intensive computation tasks in environments marked by dense infrastructure and limited resources. UVCO relies on game theory to optimise offloading decisions, leading to a notable reduction in average service costs. However, a constraint of this approach is its reliance on the availability of a UAV with substantial computing capabilities, which might not always be feasible or accessible in practical, real-world situations. In [47], Wang et al. introduced a task offloading method based on meta reinforcement learning, enabling efficient adaptation to MEC environments with a small number of gradient updates and samples, significantly reducing latency compared to baseline methods. This method's drawback is that it prioritises latency optimisation and adaptability to novel MEC environments over other aspects, such as energy efficiency or security, which are also vital for real-world MEC systems.

By analysing the existing works, it is evident that previous research studies have not given sufficient attention to several critical aspects of task offloading in edge computing environments — reliability, energy efficacy and latency reduction. Nonetheless, it has been shown that an optimal device matching strategy can greatly enhance the offloading process by efficiently assigning tasks to suitable computing devices and effectively utilising fog resources [12,28]. Only a limited number of studies simultaneously assess the influence of various factors on the performance of offloading policies such as the selection of trusted devices, SDN usage for dynamic task division and TL for reducing the training time of offloading algorithms. The proposed approach- EDITORS solves all the existing limitations by incorporating SDN, DRL and TL to develop an optimal task offloading approach which satisfies aspects such as trustworthiness (reliability), energy efficiency and latency reduction.

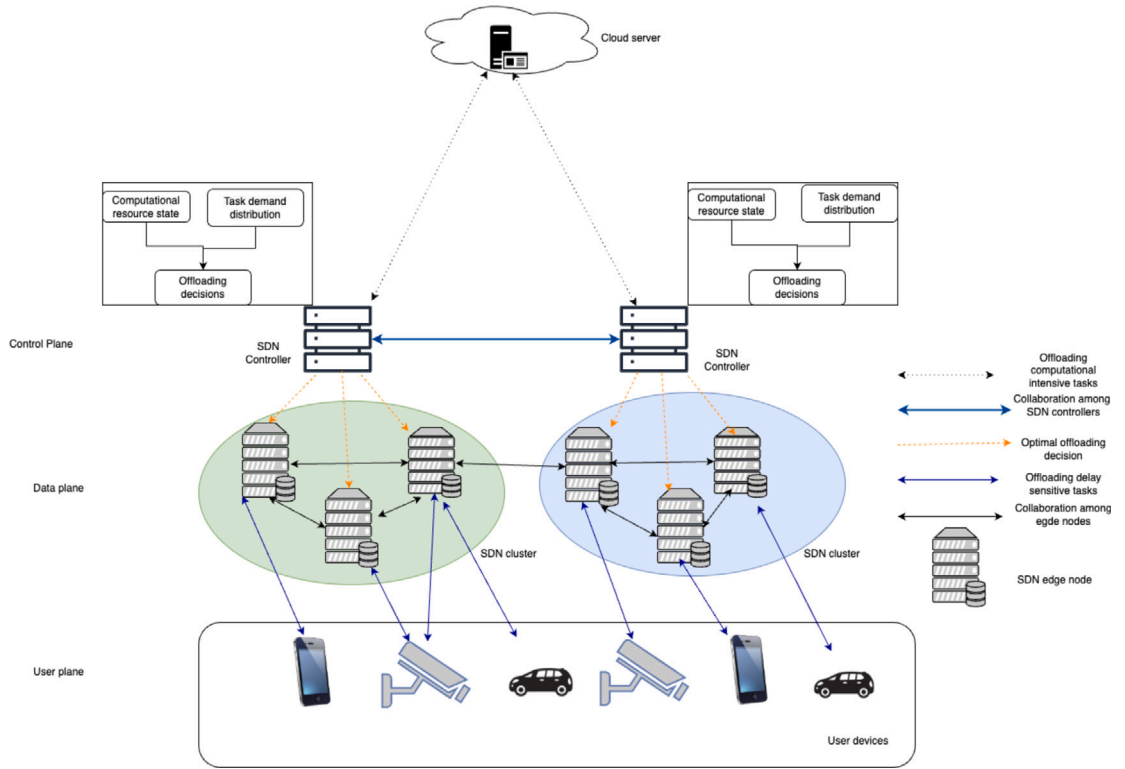


Fig. 1. Proposed distributed SDN controller architecture for dynamic partial task offloading.

3. System model

This section introduces the proposed distributed SDN controller architecture for dynamic partial workload offloading. The offloading problem formulation is then presented.

3.1. System description

A proposed distributed SDN controller architecture with three planes – user plane, data plane, and control plane – is shown in Fig. 1. Each plane has a unique function. The key principle of SDN is that networks have separate control and data planes, and this division needs to be established via an open interface [48]. In the simplest terms, the control plane decides how the network should act, the data plane implements that behaviour on individual packets, and user plane operations handle data traffic forwarding. In the user plane, the end-user devices must execute computational tasks locally or offload latency-sensitive tasks to edge nodes. A collection of SDN-enabled collaborating edge nodes forms the data plane. The control plane is made up of one or more distributed SDN controllers that create traffic flows in accordance with network rules and manage and regulate the edge nodes that make up the underlying data plane. The SDN control plane is a key element in coordinating the most appropriate task offloading choices via Deep Reinforcement Learning (DRL) in EDITORS approach. It serves as a focal point for network management, using DRL algorithms to decide when and how to offload tasks in response to real-time information on trust scores, processing speeds, and resource availability. It also dynamically reprograms data plane devices to minimise latency and improve network efficiency. The SDN user plane reflects the heterogeneous nature of network participants by encompassing a wide range of users, such as mobile phones, vehicles, and CCTV cameras. The SDN controller continuously gathers real-time network data and adjusts resource status based on data uploaded by mobile users. This allows the controller to modify the dynamic flow table and provide well-informed control instructions by taking into account task demand, vehicle proximity, and processing capabilities while determining the most appropriate offloading approach. The proposed architecture considers dynamic partial task offloading through distributed SDN controllers. In this setup, nodes can offload computational tasks to nearby nodes based on their processing capacity and task demand distributions.

The three planes' interactions are coordinated in our network architecture to guarantee effective offloading decisions. With DRL algorithms, the SDN controller in the control plane optimises task allocation based on current network conditions and is primarily responsible for managing offloading decisions for the edge nodes on the data plane. However, individual user devices or end systems are the primary decision-making level for offloading for the user plane, which includes mobile devices and end

Table 2
Notations and definitions.

Notations	Definitions	Notations	Definitions
M	Set of tasks	m	Loop variable for task
N	Set of edge nodes	n	Loop variable for edge node
s	Set of subtasks	j	Loop variable for subtask
E	Energy consumption data	w_e	Weight for energy consumption
L	Latency	w_l	weight for latency
U	Memory usage	w_u	Weight for memory usage
TS	Trust score	w_{ts}	Weight for trust score
a_m	Task size	α	Total number of subtasks assigned to edge node
z_m	Number of CPU cycles to execute one bit of the task	D_m	Total number of CPU cycles for entire task execution
B	Bandwidth	f^c	Edge processing rate
$x_m(t)$	Binary variable for task offloading decision	β	Delay constraint
H	Entropy of attribute	θ	Threshold

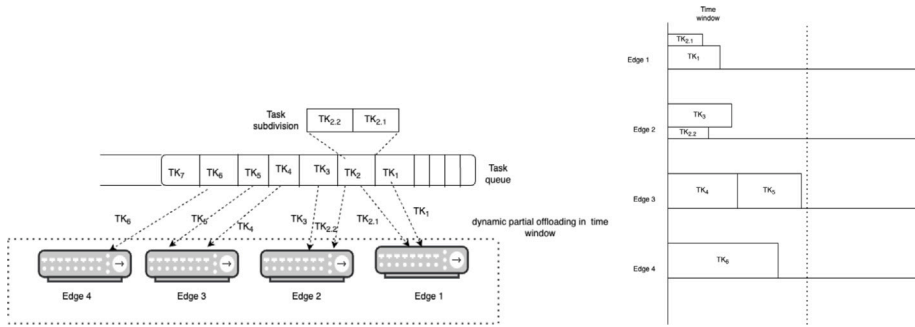


Fig. 2. Dynamic partial task offloading model that enables the division of some tasks, like task TK_2 , into subtasks, like tasks $TK_{2,1}$ and $TK_{2,2}$, in order to minimise the energy consumption and latency for task execution.

users. User devices decide whether to handle tasks locally or offload them, evaluating several factors such as their own processing power, energy availability, and task-specific needs. The SDN controller facilitates communication between the data plane and the user plane by obtaining information from the edge nodes about resource availability, processing times, and energy usage. Using this data, the controller may allocate tasks more effectively by making well-informed decisions about offloading them to the data plane. In the meantime, the SDN controller receives updates from user devices in the user plane about energy usage and resource availability. This enables the controller to optimise offloading decisions by taking these aspects into account. Thus, the user plane gives individual devices the ability to choose which tasks to offload, while the SDN controller mainly concentrates on coordinating resource management in the data plane to reduce energy consumption and task latency.

The edge nodes directly engage with end users and transmit information based on collected traffic data and queue status to the controllers, while the SDN controller assumes responsibility for aggregating information and making decisions. The system operates as follows: Initially, the SDN controller monitors network settings and edge node resources, collecting data on resource availability, processing time, trust scores, and edge node energy consumption. Using this information, the controller employs DRL algorithms to determine whether the current node should offload its requested tasks. If offloading is warranted, the controller decides how many tasks should be offloaded to a neighbouring node selected based on its task demand, energy consumption, and the number of pending tasks in its queue. Meanwhile, the edge nodes in the system execute local processing tasks and keep the SDN controller informed about their energy consumption and resource availability. After receiving optimal offloading decisions from the controller, the edge nodes execute the allocated subtasks. The major goal is to select the most appropriate offloading strategies while reducing energy consumption and task execution latency.

3.2. Task model

Let there be N edge nodes: $edge_1, edge_2, edge_3, edge_4 \dots edge_N$. The exponential distribution of computation tasks, which arrive at the task queue on average at a pace of tasks per second (tasks/s), is observed. A task set $TK = TK_1, TK_2, \dots, TK_M$, including M computational tasks, is present in the task queue at any given moment. Edge nodes can process more than one task at a time. In addition, a task TK_m with size a_m can be divided into $s = N + 1$ subtasks (i.e., TK_m can be divided into subtask, $TK_{m,j}$, where $j = 0 \dots s$), each of which can be handled by same edge node or separate edge nodes in parallel based on the resource capacity of edge nodes. The function of $s = N + 1$ indicates that each computational task can be subdivided into $N + 1$ subtasks, allowing for parallel processing, where these subtasks can be computed simultaneously on different edge nodes based on their resource capacity. The range of 0 to N ensures that all possible subtasks are considered, with $s = 0$ representing the original task and $s=1$ to N representing the subdivided tasks. When $s = 0$, it signifies that the task is indivisible and can be handled entirely by a single edge node. On the

other hand, when s takes values from 1 to N , it indicates that the task cannot be efficiently handled by a single edge node alone. In such cases, the task is divided into s subtasks. This parallelisation allows for the efficient utilisation of the available edge nodes, distributing the computational load and potentially reducing overall processing time.

Two subtasks, $TK_{2,1}$ and $TK_{2,2}$ from TK_2 , can be computed simultaneously. As shown in Fig. 2, $edge_1$ and $edge_2$ can compute two subtasks at once, for instance $TK_{2,1}$ and $TK_{2,2}$. When $s = 0$, a single edge node does the entire task. Additionally, depending on the available resources, an edge node may process several tasks at once. As illustrated in Fig. 2, $edge_2$ is capable of concurrently executing tasks TK_3 and $TK_{2,2}$ in parallel.

As seen in Fig. 2, edge nodes collaborate in this task model to carry out a variety of activities within a given time window. The model shows a flexible approach to task distribution: TK_2 , which is split into two subtasks ($TK_{2,1}$ and $TK_{2,2}$) is executed on $edge_1$ and $edge_2$ at the same time whereas TK_1 , TK_3 , and TK_6 are handled by single edge node ($edge_1$, $edge_2$, $edge_4$) respectively. As an example of its multitasking capabilities, $edge_3$ manages both TK_4 and TK_5 . This distribution pattern implies that in order to maximise resource utilisation and reduce execution time, the model carefully takes into account variables like task complexity, edge node capabilities, and potential dependencies. Queued tasks are handled according to the first-come, first-served principle (FCFS), regardless of priority.

Let the size of task TK_m be a_m and its computational complexity, z_m , representing the number of CPU cycles needed to execute one bit of the task. The total CPU cycles required to finish the entire task can be expressed as $a_m z_m$. For a given edge node, $edge_n$ with α_n subtasks assigned to it, the total number of CPU cycles required to complete these subtasks is $\alpha_n a_m z_m$.

The following assumptions are made in this study: (1) subtasks can be generated from a task (2) subtasks arrive in edge nodes simultaneously [49–51] (3) the edge nodes do not have any major communication and transmission overhead. Edge nodes often operate with resource constraints, such as limited memory, processing power, and energy [52–54]. These limitations could result in a design that reduces transmission and communication overhead. (4) there is no interdependence of data between subtasks (5) The output will be accessible on every node and the outcomes obtained from all nodes have to be merged in order to generate the final output. Table 2 presents the various notations used in this paper.

3.3. Problem formulation

The goal of this study is to develop a DRTL approach that enables the offloading of each individual task as it joins the task queue at the edge. This is done with the purpose of diminishing task execution latency and reducing energy consumption, thereby ensuring the efficiency, adaptability, trust management, and reliability aspects of task offloading. The approach uses an SDN controller to explore dynamically dividing the tasks and taking advantage of the parallel computing of subtasks among multiple edge nodes to achieve the objectives.

Consider a MEC system with N edge nodes, $Edge = edge_1, edge_2, \dots, edge_N$ and TK be the set of M tasks, $TK = TK_1, TK_2, \dots, TK_M$ is present in the task queue. Let us consider about the following:

- Task offloading decisions are often modelled using binary variables because they provide a straightforward and effective way to capture the decision-making process. Within the domain of task offloading, a binary variable usually takes on one of two values: either 0, denoting the option to carry out the entire task locally on the device, or 1, denoting the decision to offload the entire task to the edge node. The decision-making process is made simpler by this binary representation, which turns it into a discrete optimisation problem. The binary character of these variables is important because it requires one to make a clear and definitive decision between local execution and offloading. This makes the decision-making process feasible for real-time edge computing scenarios and computationally feasible. The task offloading decision is formulated as a binary variable $x_m(t)$ in respect of time t .

$$x_m(t) = \begin{cases} 1, & \text{if } TK_m \text{ is offloaded to } edge_n \\ 0, & \text{otherwise} \end{cases}$$

The binary decision variable $x_m(t)$ in the model that is being presented acts as an abstraction to indicate whether the task is offloaded in the whole to a certain edge node, $edge_n$ at a given time t . The task's high-level offloading decision can be more easily modelled with this binary representation. Although it is recognised that every task TK_m can be split up into several subtasks that could be offloaded to various edge nodes, $x_m(t)$ streamlines the way the offloading choice is represented at the task level. The allocation of subtasks to different edge nodes is captured through other variables and mechanisms in the model, such as data size.

- The total energy consumption for offloading task m to edge node n ,

$$E_m(n) = E_m^{upload}(n) + w_e D_m$$

where $E_m^{upload}(n)$ - energy consumption to upload task m from user device to edge node n , w_e -weight of energy consumption factor at edge node n and D_m -total number of CPU cycles to execute task TK_m .

- The total latency L_{nm} represents the combined time it takes for a specific task m to be offloaded, transmitted, and processed in edge node n . There are two primary components to this calculation: First, D_m/B_{nm} where D_m indicates the task's computing load in terms of CPU cycles and B_{nm} the user's allotted bandwidth. Using the allotted bandwidth, this component calculates how long it takes to offload the task to the edge node. Secondly, D_m/f^c where f^c is the edge node processing rate, indicating how quickly the edge node can execute the task that has been received. This part determines how long it will take for edge node to finish the task. The total latency,

$$L_{nm} = \frac{D_m}{B_{nm}} + \frac{D_m}{f^c}$$

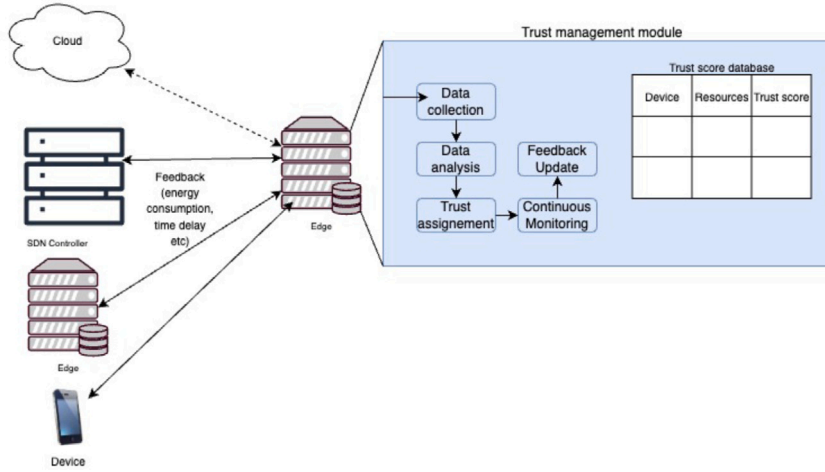


Fig. 3. Feedback-based trust management.

- Cost function is the weighted sum of the latency and energy consumption of the task computation in our system.

$$Cost\ function, \mathbb{C} = \sum_{n=1}^N \{ (w_e * \sum_{m=1}^M E_m(n)) + \beta * \max L_{nm} \} \quad (1)$$

where β is the delay constraint.

- Trust function determines the total trust of the n selected edge nodes.

$$Trust\ function, \mathbb{T}\mathbb{F} = \sum_{n=1}^N (w_{ts} * TS_n) \quad (2)$$

where TS is the trust score and w_{ts} is the weight of trust score.

- The main objective of the proposed task offloading is to minimise the cost function and maximise the trust function which satisfy reliability and resource usage optimisation characteristics.

4. Proposed feedback-based trust management using LSTM

The primary goal is to design a task offloading system that incorporates trusted edge nodes while prioritising energy efficiency, timeliness, reliability, adaptability, and outperforming existing task offloading methods in terms of the quality of the task offloading plan. This is important to take into account in real-world scenarios like the person re-identification applications to identify those nodes that (1) do not allocate enough resources to perform the person re-identification tasks or (2) perform more computations by integrating some malicious intentions into their local execution. The most appropriate trust technique should determine whether any consumption of the edge node is greater than or less than the computed abnormal limit for any feasible consumption.

The feedback method can be used to evaluate the reliability and performance of the edge nodes. The proposed feedback-based trust management gathers feedback information from the cloud, other edge nodes, and user devices about an edge node based on prior experience in terms of latency, energy consumption, and memory usage. The SDN controller can then decide which edge nodes to employ for task offloading and can assign various levels of trust to various devices based on this information. A deep learning model utilising LSTM can be trained with feedback data to predict trust scores based on a variety of variables, such as energy usage, latency, and memory usage. The SDN controller considers variables like energy efficiency, dependability, and user experience when choosing the most appropriate nodes for offloading tasks based on the predictions of the model. In order to increase the consistency of the offloading decisions over time, the trust scores are regularly updated depending on the feedback data. MEC systems can enhance service quality, lower the chance of security failures, and ensure offloading activities are completed successfully and efficiently by utilising the proposed feedback method for trust management. Fig. 3 presents the proposed feedback-based trust management using LSTM.

The trust score of the edge devices for current task offloading is generated using the memory consumption of previous task executions, which is computed from the dataset. We may evaluate the dependability and efficiency of the edge devices in managing new task offloads by looking at memory usage patterns from earlier workloads. The trust score is determined by combining memory consumption data with energy consumption, latency, and their corresponding weights. As a result of this trust score, tasks are decided to be offloaded to edge devices.

The algorithm for trust score calculation using LSTM is explained in detail in the following.

Algorithm 1 Trust score calculation using LSTM.

Input: E be the set of energy consumption data, L be the set of latency data, U be the set of memory usage data for the edge devices, X is the input features matrix, Y is the target label matrix, w_E is weight for energy consumption data, w_L is weight for latency data and w_U is weight for memory usage data.

Output: TS , trust scores of edge nodes.

- 1: Split the data X and Y into training set (X_{train}, Y_{train}) and validation set (X_{val}, Y_{val}).
- 2: Train the LSTM model on X_{train} and Y_{train} for a predetermined number of epochs.
- 3: For each sample in the validation set X_{val} :
- 4: Normalise the data: E , L and U
- 5: Compute the entropy, H for the weight calculation using Shannon's entropy method:
- 6: $H_E \leftarrow w_E * \log_2(w_E)$
- 7: $H_L \leftarrow w_L * \log_2(w_L)$
- 8: $H_U \leftarrow w_U * \log_2(w_U)$
- 9: Calculate the final weight for each attribute:
- 10: Final weight for energy consumption, $w_e \leftarrow \frac{1-H_E}{3-\sum_1^3 H_E}$
- 11: Final weight for latency, $w_l \leftarrow \frac{1-H_D}{3-\sum_1^3 H_D}$
- 12: Final weight for memory usage, $w_u \leftarrow \frac{1-H_M}{3-\sum_1^3 H_M}$
- 13: Calculate the trust score,
 $TS \leftarrow w_e * E + w_l * L + w_u * U$
- 14: Calculate the Mean Absolute Error between the predicted and actual trust scores (Y_{val}) to determine the threshold, θ .
- 15: Repeat steps 4 through 14 once again to constantly adjust the trust scores for each edge node when feedback data is updated
- 16: Trust score, $TS = TS \leq \theta, \text{untrusted}; TS \geq \theta, \text{trusted}$
- 17: Return TS to the trust score database

- Data Pre-processing: The values of an input feature matrix are scaled to fall between 0 and 1 using the min-max normalisation approach [55]. This is achieved by normalising each value within the feature's range, followed by subtracting the minimum value of the feature from each data point.
- Determine the final weights using Shannon's entropy method:- Dynamically calculating weights based on the given criteria is done using Shannon's entropy method [56].
 The entropy of the i th attribute (e.g. energy consumption, latency and memory usage) is

$$H_i = w_i * \log_2(w_i) \quad (3)$$

where w_i is the initial weight of the attribute i .

The final weight for energy consumption,

$$w_i = \frac{1 - H_i}{o - \sum_1^o H_i} \quad (4)$$

where o is number of attributes and $i = 1, 2..o$.

- Trust score calculation using weighted sum: Trust score is calculated using the weighted sum of selected attributes and final weights.
 Trust score,

$$TS = w_e * E + w_l * L + w_u * U \quad (5)$$

where w_e is weight for energy consumption data, E be the set of energy consumption data, w_l is latency data, L be the set of latency data, w_u is weight for memory usage data and U be the set of memory usage data for the edge devices.

- Calculate the threshold value and check whether the edge node is trusted or not. Mean Absolute Error (MAE) [57] is a metric that assesses how closely predicted values match actual outcomes. When utilising MAE to calculate the threshold, the term "actual value" refers to the ground truth trust values, which are derived from the dataset. They are the benchmarks for trustworthiness. Conversely, the term "predicted value" denotes the predicted trust values generated by the LSTM-generated trust prediction model. The model's estimates of trust for each relevant sample in the dataset are represented by these predicted values. The average absolute difference between these actual and predicted trust values is then measured by the MAE, which provides a quantification of the errors in trust prediction. When validating an LSTM model, it is specifically employed in trust

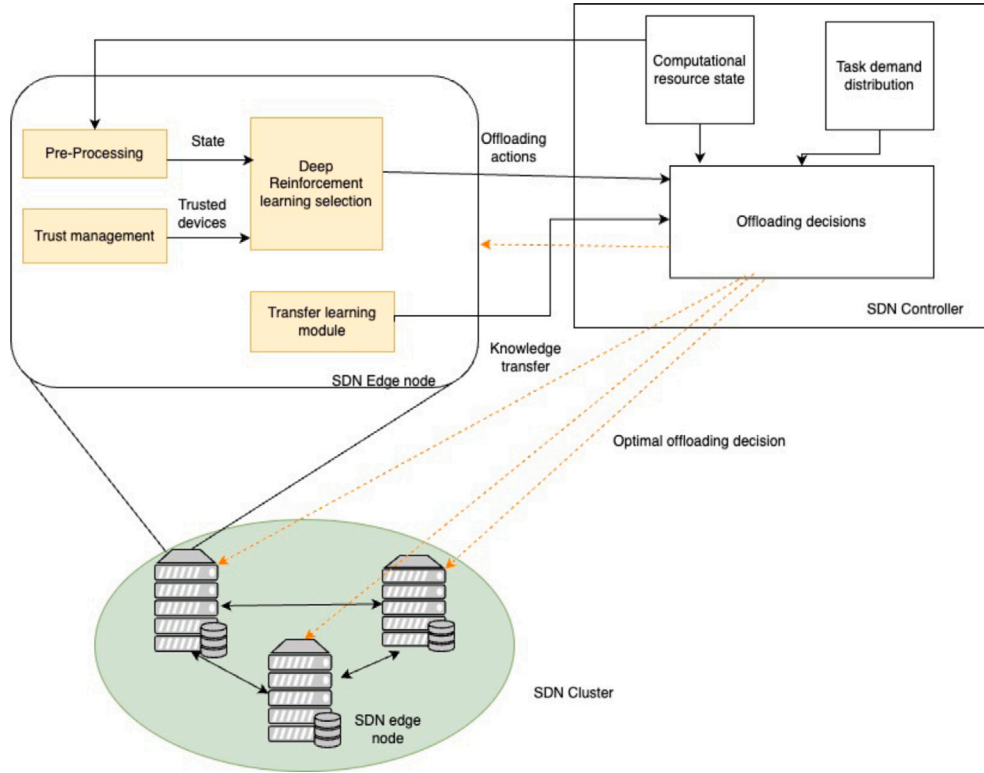


Fig. 4. EDITORS- Proposed DRTL framework for task offloading.

management to establish the threshold point for determining how well predicted trust scores match actual trust scores.

$$Threshold, \theta = \frac{actual\ value - predicted\ value}{n} \quad (6)$$

This threshold value can be used to select the trusted edge node by checking the condition: If the edge node's trust score is higher than or equal to the threshold value, it is trusted; otherwise, it is untrusted.

5. EDITORS- proposed DRTL-based task offloading algorithm with distributed SDN controller

The main objective of this study is to integrate DRL with TL to create a DRTL for dynamic partial task offloading within distributed SDN controller-enabled MEC systems. This DRTL framework is deployed in each edge node of a MEC system and collaborates with distributed SDN controllers and other edge nodes to access state information and trust scores. This whole framework is named EDITORS-Energy-aware DynamIc Task Offloading using DRTL in SDN-enabled edge nodes. EDITORS is used to produce the most appropriate task offloading among edge nodes while considering the least amount of energy and execution latency. The framework's goal is to assign tasks quickly and effectively based on the dependability and performance of edge nodes, improving overall system performance in terms of resource utilisation. Through the use of this DRTL, SDN controllers can assign computational tasks to the most appropriate edge nodes for execution based on the optimal offloading strategies produced by the DRL module. The TL module speeds up the convergence rate of DRL by enabling the recently included devices to quickly adapt to current offloading techniques. The primary benefit of the EDITORS is the ability to choose the edge nodes that will offload dynamically while using the least amount of energy and time. Fig. 4 presents the proposed DRTL framework for task offloading. The main modules in the EDITORS approach are explained below:-

- **Pre-processing:-** In DRTL, the pre-processor collects observations on the state of the devices using a distributed SDN controller once a round is finished. These observations include energy usage, latency, and network bandwidth between each edge node and the user device. The pre-processor normalises the training time per iteration.
- **Trust management:-** The proposed feedback trust management within DRTL framework generates trust scores that indicate the trustworthiness of each edge node. The set of trusted devices is obtained and fed to the DRL selection module.
- **Deep reinforcement learning selection:-** Using state data and a collection of trusted devices, the trained DRL agent in this module applies a fully connected neural network to develop the most appropriate offloading decision action.

- **Transfer learning module:-** Transfer learning enables edge nodes to transfer pre-trained models from currently existing nodes to newly added nodes, shortening the DRL training period. Thus, by including TL in DRL, edge nodes can exchange knowledge without starting from scratch. This is essential to reduce the amount of time required for DRL training on each edge node and to address concerns about data scarcity.
- **Offloading decisions:-** The offloading actions created by DRL agents are reviewed in this module along with pre-trained models from the TL module to determine the most appropriate offloading actions in the SDN controller for decision-making and scheduling tasks to the most appropriate edge node.

5.1. Algorithm for subtask division in dynamic partial task offloading by SDN controller

Algorithm 2 Subtask Division in dynamic partial task offloading.

Input: N be the set of edge nodes, E_n be the energy consumption of a edge node n , M be the set of tasks, s be the number of subtasks, E_m be the energy needed for execution of a task m and SDN be the SDN controller.

Output: A mapping MP: $n \leftarrow m$, where MP (m) represents the offloading of task m to edge node n .

- 1: Initialise a mapping MP of edge nodes to tasks or subtasks.
 - 2: Create an empty list in SDN to hold the subtasks and initialise it: $subtasks = []$
 - 3: If s is equal to 0, do the following:
 - 4: Create a single subtask consisting of the entire task: $subtasks.append([E_n])$
 - 5: Return the list of subtasks
 - 6: Else:
 - 7: For each subtask index from 1 to s , do the following:
 - 8: Dynamically divide the task into subtasks:
 - 9: Calculate the energy consumption of each edge node for the subtask: $E_{subtask}(n) \leftarrow E_m(n) - E_m(subtask[m])$ for all n in N .
 - 10: if $(E_m(n) \geq E_{subtask}(n))$:
 - 11: Assign subtask to edge node n .
 - 12: Update the energy consumption of n : $E_m(n) \leftarrow E_m(n) - E_m(subtasks[m])$
 - 13: Add the assignment to the mapping MP: $n \leftarrow (subtasks[m])$.
 - 14: Return the mapping MP as the final task or subtask assignment to edge nodes and SDN
-

Algorithm 2 shows the subtask division and assignment to edge nodes implemented inside the SDN controller, which gathers data from the underlying SDN routers about the available edge node capabilities, including their energy consumption and execution time. Based on the energy capacities of edge nodes, the algorithm attempts to allocate tasks or subtasks to those nodes. If a task has just one subtask, the algorithm constructs a single subtask that represents the whole task and assigns it to the most appropriate edge node. If there are numerous subtasks, the algorithm advances by continuing through each subtask. It determines whether an edge node's energy consumption is adequate for carrying out the subtask. Upon confirmation, the edge node receives the subtask, and its energy consumption is modified. The mapping function maintains the assignment and links the edge node to the appropriate subtask. Each subtask goes through the same procedure, culminating in a full mapping of tasks or subtasks to the designated edge nodes. To shorten the processing latency, the DRL algorithm chooses the edge nodes depending on their energy consumption and execution latency. The controller keeps track of a log dataset of the current node's processing and energy consumption and updates it continuously after tasks have been assigned to their appropriate nodes.

The SDN controller uses Algorithm 2 to dynamically divide each task into subtasks. The proposed method divides tasks appropriately by using real-time data from SDN routers on edge node capabilities, such as energy consumption and execution time. The proposed strategy powers this dynamic segmentation and assignment procedure, which is an essential part of the task offloading approach. The algorithm's dynamic structure guarantees its capacity to adjust to real-time circumstances, as it relies its segmentation decisions on the capabilities and present state of edge nodes. The controller continuously updates a log dataset of the processing and energy consumption of the current node, and the algorithm takes execution delay and energy consumption into account. This proposed method, which is driven by the DRTL algorithm, seeks to reduce processing latency and improve task offloading system's overall effectiveness.

5.2. DRTL-based task offloading algorithm

The proposed DRTL offloading technique is implemented using a multi-layered neural network [58] that uses the network's parameters to describe actions, all of which are input into the network itself to create an aggregate action vector for the state. Eq. (1) defines the cost function as a combination of edge node states and actions. The offloading policy O is defined as tuple $\langle S, A, R \rangle$ where:

- S - Set of states in the system
- A - Set of actions over all the edge nodes
- ψ - Reward function: $S \times A \rightarrow \psi$

The state of the edge devices can be represented as $[M, N, E, L, TS]$ where M be set of tasks to be offloaded, N be set of edge nodes available for offloading, E be energy consumption values of edge nodes, L be the execution latency and TS be the trust scores of edge nodes. Since trust score and latency are dynamic, they may vary from one state to another. The action for each task is given by

$$A_m = (x_m, T_m, E_m, C_m) \quad (7)$$

where x_m is the task offloading decision in which the value is 1 if task is offloaded to edge, T_m is the total execution time for task offloading, E_m is the energy consumption needed for task offloading and C_m is the total cost for offloading. The SDN controller uses a Q-learning algorithm to simulate a learning agent that links environmental states to action. To maximise a reward over time, the agent perceives these actions as shifting from one state to another. Even if an agent does not finish the knowledge of rewards and state transition functions, the SDN controller specifically chooses these actions at runtime. The reward Ψ for the node n is a function of state $st \in S$ and action $a \in A$ and is computed as follows:

$$\Psi = \sum_{n=1}^N (w_l \sum_{m=1}^M \gamma L_{nm} + w_e \sum_{m=1}^M \gamma E_{nm} + w_{ts} \sum_{m=1}^M \gamma TS_{nm}) \quad (8)$$

where w_l is the weight for latency, w_e is the weight for energy consumption, w_{ts} is the weight for trust score, γ to check if task is executed or still in task queue.

The objective of Algorithm 3 is to identify the most appropriate offloading strategy for dynamic task offloading in terms of execution latency and energy consumption of edge nodes. It employs a Q-learning methodology, and a neural network called the Q-network [59]. The Q-network is given random weights as the process begins. It then does the following actions for each episode: creating the initial state, choosing an action based on the epsilon-greedy policy [60], carrying out the action and observing the new state, determining the reward based on a cost function, and modifying the Q-value for the state-action pair. The epsilon-greedy policy (ϵ) is a strategy in RL, in which an agent must decide which actions to do in the environment to maximise its cumulative rewards [60]. The policy strikes a balance between the investigation of potentially superior actions and the utilisation of presently understood most appropriate actions. We choose the optimum course of action using a probability of $1 - \epsilon$, changing the target network's output in accordance with the reward while also updating the parameters in each state and enforcing a new policy. The energy consumption, execution latency, and trust score of the edge nodes are weighted combinations used to calculate the Q-value. A higher Q-value means the chosen edge server is more suited for the current subtask. The Q-value evaluates how well the edge server fits the current edge task. At the end of the episode, the algorithm keeps updating the state. When a new edge node is contacted, the pre-trained Q-network weights from an earlier edge node are transferred and modified. The algorithm then decides on the most appropriate offloading policy based on the Q-values after an adequate number of episodes. As a result, the output is the most appropriate possible policy. This algorithm uses DRTL and the Q-network to make the optimum offloading choices while taking into account a number of variables, including energy consumption, execution time, and edge node trust scores.

Algorithm 3 DRTL based task offloading.

Input: M be the set of tasks to be offloaded, N be set of edge nodes available for offloading, SDN be the SDN controller, E be energy consumption values, L be the execution latency and TS be the trust scores of edge nodes and w_e , w_l and w_{ts} be the weights.

Output: Optimal offloading policy O .

- 1: Use random weights to initialise the Q-network.
- 2: For each episode:
 - 3: Set the initial values of state S to $[M, N, E, L, TS]$.
 - 4: Using the ψ , SDN choose action A depending on the current state.
 - 5: Carry out action A using Equation 7 and see the changed state S' .
 - 6: Determine the reward ψ using Equation 8 based on the cost function for the new state S' .
 - 7: Compute the Q -value for the state-action pair (st, a) :

$$Q \leftarrow w_e * E(n) + w_l * L(n) + w_{ts} * TS$$
 - 8: As you go through steps 3 to 7, update the state $S \leftarrow S'$ until the end of the episode.
 - 9: If n is a new device:
 - 10: Transfer the pre-trained Q-network weights from existing edge node n .
 - 11: Update the Q -values during training with the knowledge transfer received.
 - 12: SDN choose the most appropriate offloading policy O based on the Q -values after an adequate number of episodes.
 - 13: Return the optimal offloading policy O .

6. Performance evaluation

This section encompasses a dataset description, offering intricate information about the datasets utilised, and simulation settings, which includes the evaluation criteria, parameters, and details regarding the simulation platform.

6.1. Dataset description

The dataset used for edge nodes is the MEC dataset collected from the MEC-simulator [61]. This simulator's main goal is to calculate the overall time and energy consumed when processing activities for a specific application from start to finish [61]. We used data from this dataset for energy consumption, total execution time, and memory usage metrics to simulate offloading experiments.

PETA is the dataset primarily used for person re-identification applications. The PEdesTrian Attribute dataset (PETA) [62] is a dataset for remotely determining pedestrian characteristics including gender and apparel. It applies to scenarios involving video surveillance where getting detailed shots of the body and face is challenging. The collection consists of about 19,000 pedestrian images with 65 attributes (sixty-one binary and four multi-class). In those images, there could be 8705 people [62].

6.2. Simulation settings

The trust score was calculated, and the preprocessing task was completed in this experiment on Google CoLab using Python library packages like Pandas, Numpy, Scikit-Learn, Matplotlib, and Keras. LSTM cells were integrated into EDITORS due to their proficiency in managing sequential data, which is well-suited to the inherent characteristics of person re-identification tasks. With LSTM cells, dropout, and dense output layers are utilised to determine the selection of trustworthy devices based on their trust scores. The model structure includes one input layer, two LSTM cells and one dense layer. LSTM (Long Short-Term Memory) is a form of recurrent neural network frequently employed for processing sequence data [63]. After the LSTM cells, there is just one Dense layer in the model. Dense layers are those in which every neuron is linked to every neuron in the layer above. When the required number of units is supplied, this layer serves as the output layer. After the dense layer, a dropout rate of 0.20 is also applied. Dropout is a regularisation approach that helps avoiding overfitting by randomly setting a portion of input units to 0 during training [63]. The Rectified Linear Unit (ReLU), sigmoid activation functions, and Adam optimiser were also used in the model. We used the sigmoid and Rectified Linear Unit (ReLU) activation functions because they have been shown to be successful in deep neural networks. The Adam optimiser was selected because it is widely used for neural network optimisation and has the ability to modify the learning rate, which tends to accelerate convergence. The training and testing sets of the MEC dataset were divided into them in a ratio of 70:30, respectively. The ratio of 70:30 between the training and testing sets in our dataset split was designed to achieve a balance between evaluating model generalisation and guaranteeing a sizable enough training set for efficient learning.

The proposed DRTL approach is based on simulation tests performed on Google CoLab with Python library packages like Pandas, Numpy, Scikit-Learn, Matplotlib, and Keras with the SDN environment. The Ryu-SDN controller [64] and Mininet version 2.2.2 [65] are used to simulate the SDN environment on an Intel XeonGold 5118 processor with 48 cores. The number of CPU cycles to process one bit of data is 500 cycles/bit. Global topology discovery, task partitioning, load balancing, and monitoring functions are all handled by the Python-based Ryu SDN controller. A few of the existing offloading techniques without SDN were compared to the performance of the proposed DRTL-SDN. Table 3 illustrates the settings for the simulation parameters. The parameter values were chosen to create a trade-off between simulating the various aspects of the edge-computation environment, effective learning convergence, and available computational resources. With five edge nodes, we can build a small-scale edge computing network that is manageable for initial testing without excessive processing demands. Our goal is to simulate a moderate workload with 10 tasks, offering a variety of task scenarios for a thorough evaluation. Due to task fragmentation, the maximum number of subtasks is set at five, which replicates real-world situations where jobs can be separated into smaller groups for effective offloading. A comparatively moderate decay rate of 0.995 guarantees a progressive learning rate reduction for fine-tuning during training, while a learning rate of 0.8 is chosen to speed up the learning process. An optimal balance between model stability and efficiency is achieved with a batch size of 32. We chose 1000 episodes to enable deep learning, which is especially important for iterative optimisation and reinforcement learning. The selection of these parameters is customised to fulfil our research goals while adhering to the limitations of simulation precision and processing power.

7. Results and discussion

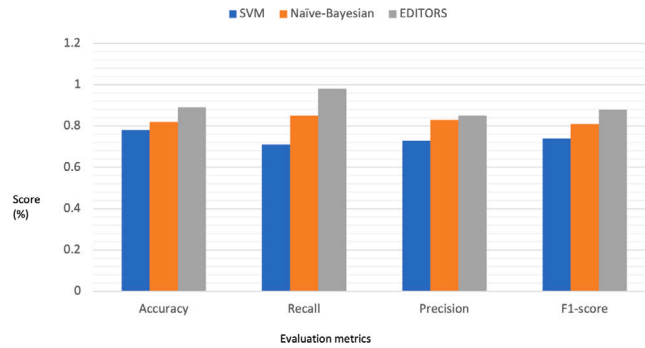
The first section compares the performance of the proposed model with other machine learning techniques and describes the experiment results analysis of the trust score calculation using LSTM. The performance of the proposed DRTL algorithm is examined in the following subsection in simulations where the system is thought to be used for a person re-identification application where tasks are divided and offloaded among edge nodes. This section conducts simulated experiments to investigate the offloading of tasks to most trusted edge nodes within an SDN framework. The experimental findings can be broken down into six categories: (i) trust score calculation experiments and comparison with existing methods; (ii) re-weighting experiments by adding more weights to latency; (iii) re-weighting experiments by adding more weights to memory usage; (iv) experiments to obtain the most appropriate offloading policy by EDITORS; (v) experiments to analyse number of edge nodes on total cost and (vi) experiments to analyse number of edge nodes on latency.

Table 3
Simulation parameters.

Parameters	Value
Number of edge nodes	5
Number of tasks	10
Maximum number of subtasks	5
Learning rate	0.8
Batch size	32
Decay rate	0.995
Dropout rate	0.20
Training data	70 percent of the selected dataset
Testing data	30 percent of the selected dataset
Number of CPU cycles	500 cycles/bit
Weighing coefficients for energy consumption	0.5120
Weighing coefficients for latency	0.1935
Weighing coefficients for memory used	0.3010
Threshold	0.6984
Number of episodes	1000

Table 4
Trust scores of six edge nodes.

Edge node	Energy consumption (J)	Latency (ms)	Memory used (MB)	Trust score
A	7040	80	50	0.87
B	7040	100	60	0.75
C	11 000	120	55	0.98
D	7040	90	70	0.83
E	11 000	110	65	0.92
F	7040	200	200	0.53

**Fig. 5.** Comparison of evaluation metrics among different machine learning algorithms-giving more weight to energy consumption (0.5, 0.2, 0.3).

7.1. Trust score calculation experiments and comparison with existing methods

This paper examined the factors of energy consumption, latency, and memory usage to determine whether the device can be trusted or not. Shannon's entropy approach was used to compute the weights of each factor. Data preparation was done as the initial stage, which normalised the decision matrix to bring the data into the same range. Equations (3) and (4) were then applied to compute the entropy value and the final weights. As data or sample sizes change, weights change dynamically. The various dataset sizes have been dealt with using the entropy approach. The outcomes revealed that by employing 20% of the dataset size, an energy consumption weight of 0.5120, a latency weight of 0.1935, and a memory usage weight of 0.3010 had been achieved.

Eq. (5) and Eq. (6) were used to determine the trust score after getting the weights of the factors and comparing them to the threshold. Any result beyond the threshold (0.6984 obtained using Eq. (6)) was considered as trustworthy, and any value below it as untrustworthy. The calculation of the trust score for six edge nodes is shown in Table 4. By looking at this table, for instance, the trust score of A (0.87) is higher than threshold, indicating that the node is trusted and the trust score of F (0.53) is less than threshold, indicating that the node is untrusted. The trust score was determined using the same mechanism across the entire dataset and was utilised as an input in the next stage to generate a prediction model for trust score using LSTM.

Next, a trust score prediction model is generated using the LSTM and validated using 1000 epochs on a dataset in the 70:30 ratio. The proposed trust score prediction model is benchmarked against the most common and recent machine learning methods. Fig. 5 depicts the comparison with existing methods for trust score calculation with weights (0.5,0.2,0.3) for energy consumption, latency and memory usage. The proposed model using LSTM achieved better accuracy of 89% than existing methods: Naive-Bayesian (82%)

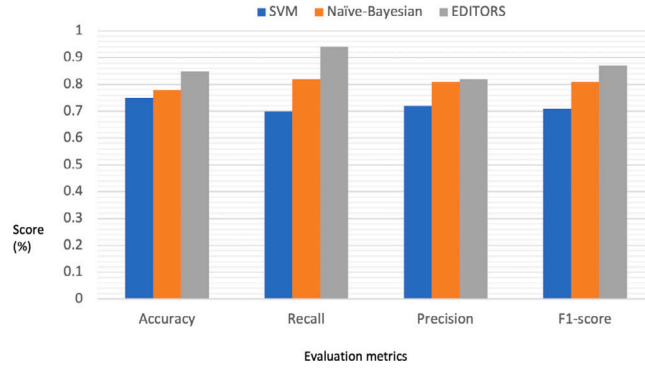


Fig. 6. Comparison of evaluation metrics among different machine learning algorithms-giving more weight to latency (0.3, 0.5, 0.2).

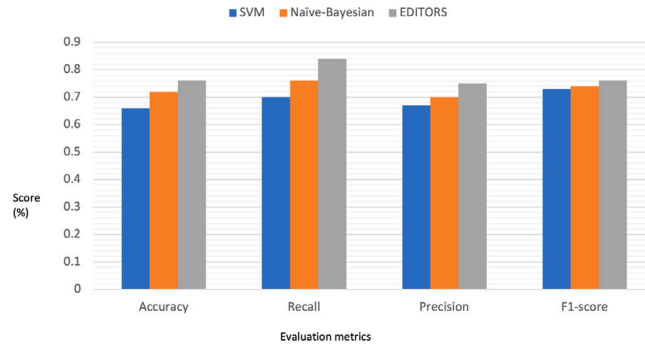


Fig. 7. Comparison of evaluation metrics among different machine learning algorithms-giving more weight to memory usage (0.2, 0.2, 0.6).

and SVM (78%). This clearly states that LSTM is the most appropriate machine learning algorithm to predict trust score of edge nodes with better accuracy than other machine learning methods. The trust score prediction using the proposed model (166 s) is faster than Naive-Bayesian (284 s) and SVM (450 s). This demonstrates that the LSTM model outperforms SVM and naive Bayesian methods in detecting trusted vs untrusted edge nodes in terms of energy consumption, latency and memory usage for most appropriate task offloading decisions.

7.2. Experiment using re-weighting by adding more weights to latency

We also conducted re-weighting experiments to find out which factor plays a major role in deciding the trustworthiness of the edge node. For this, the above-mentioned procedure is repeated with varying weights of factors including energy consumption, latency and memory usage. Fig. 6 depicts the comparison with existing methods for trust score calculation with weights (0.3, 0.5, 0.2) for energy consumption, latency and memory usage. This demonstrates that adding more weights to latency could lead to decreased overall accuracy or performance, and execution time for re-weighting computations should be longer.

7.3. Experiment using re-weighting by adding more weights to memory usage

Fig. 7 depicts the comparison with existing methods for trust score calculation with weights (0.2, 0.2, 0.6) for energy consumption, latency and memory usage. This demonstrates that increasing the weights assigned to memory usage may cause a little difference in overall accuracy, and that the computations for re-weighting should take longer overall. As shown in Fig. 5, the primary factor determining a node's trustworthiness is its energy consumption, which achieves higher accuracy and is also less time intensive. Nodes that are over or under-utilised will make the system less reliable and more vulnerable. These re-weighting experiments prove that varying the weights as mentioned in Figs. 6 and 7 did not affect the superiority of the proposed LSTM method over the SVM and Naive-Bayesian methods. Fig. 8 depicts the comparison of training time among different machine learning algorithms by varying weights for energy consumption, latency and memory usage.

7.4. Experiments to obtain most appropriate offloading policy by EDITORS

EDITORS improves energy efficiency by running an SDN-enabled IoT network with less energy consumption, as shown in Eq. (1), and by treating all nodes as battery-powered. Through the training of the agent over 1,000 episodes, the SDN controller evaluated

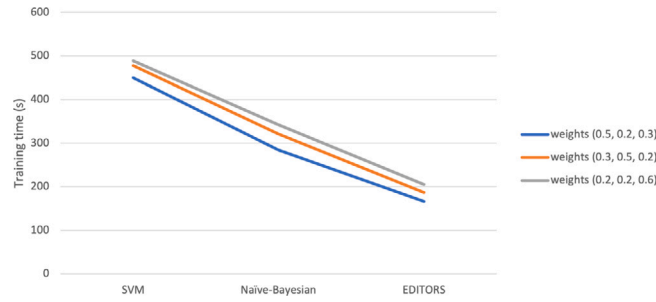


Fig. 8. Comparison of training time among different machine learning algorithms.

Table 5
List of edge nodes.

Edge node	Energy consumption	Trust score
A	7040	0.87
B	7040	0.75
C	11 000	0.98
D	7040	0.83
E	11 000	0.92

Table 6
List of person re-identification tasks.

Tasks	Energy consumption
Task1	29 300
Task2	10 589
Task3	20 689
Task4	4542
Task5	13 260
Task6	12 262
Task7	10 589
Task8	20 632
Task9	17 450
Task10	18 301

the energy efficiency of the proposed solution. As a result, the agent will be able to schedule ten tasks for five edge nodes with limited energy. The DRTL agent in the proposed approach maintains an improved energy consumption among edge nodes compared to the existing work during the most appropriate offloading plan strategy of these tasks.

Table 5 lists the five edge nodes with energy consumption rate and trust score calculated using Eq. (3). The measured energy consumption rate varies depending on the required energy to execute the person re-identification tasks. Table 6 lists the energy needed for each task. Table 7 lists several tasks, their subtasks, their energy consumption (in Joules), and the specifics of how long each task took to complete on each node (including arrival time, waiting time, completing time, and execution time). Task 1 will use a total of 29,300 J of energy when it starts at 0 ms. This task has been broken down into four separate subtasks, each assigned to a different node for execution. With a 11,000 J energy load allocated to Node C, subtask 1.1 starts immediately upon arrival and is finished in 75 ms. Similar to the last task, subtask 1.2 is given to Node E and requires the same amount of energy (11,000 J), finishing in 75 ms. The energy consumption of subtask 1.3, assigned to Node A, is 7040 J, and it completes in 60 ms. Finally, subtask 1.4 on Node D completes execution in 25 ms while using 260 J of energy. Task 2 enters the system at 0 ms beginning time and consumes 10,589 J in total. This task is divided into two separate subtasks, each assigned to a different node for execution. The energy consumption for Subtask 2.1, which is assigned to Node D, is 6780 J which run parallel with task 1.4. When it arrives, it instantly begins to run and ends successfully after 50 ms. In addition, Node B is given the duty to execute Subtask 2.2, which has an energy need of 3809 J. Similar to the preceding subtask, it starts working immediately after arrival and completes in 40 ms.

Fig. 9 depicts the task subdivision among edge nodes based on trust score and energy consumption and is explained as follows: Due to its high trust score of 0.98 and remaining energy consumption of 11,000 J, Edge node C is chosen for the first subtask allocation (Task1.1). The next trusted node is E and Task 1.2 is allocated to it, then Task 1.3 to A and Task 1.4 to D. We will search for another task that can be assigned based on remaining energy consumption and trust score as Edge D is capable of performing many tasks in parallel. Task 2.1 can be assigned to D in parallel with current task 1.4. This process of subtasks allocation among edge nodes and execution in parallel continues which helps to minimise the overall energy consumption of SDN-enabled edge nodes and total task execution time is also reduced due to parallel execution. Due to the parallel execution of multiple tasks, the failure rate is also lower and the waiting time in the task queue is also minimised.

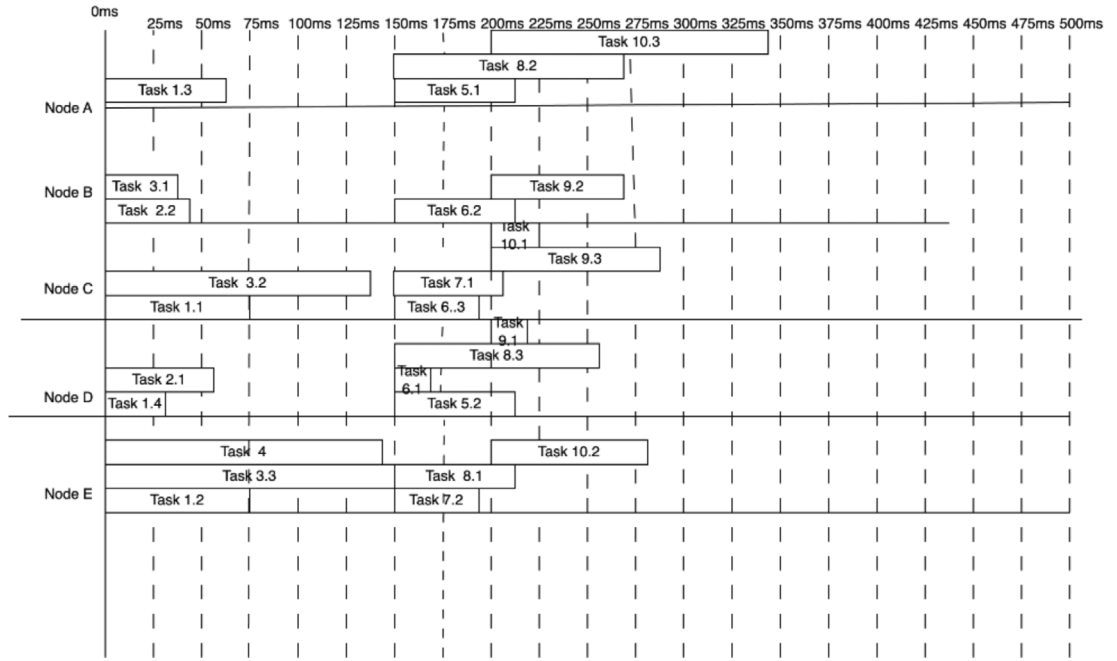


Fig. 9. Task subdivision based on trust score and energy consumption of edge nodes.

Table 7

Edge node assignment of tasks with execution details.

Tasks	Subtasks	Edge nodes	Arrival time	Waiting time	Finishing time	Execution time
Task1 (29300 J)	Task 1.1	Node C (11000 J)	0 ms	0 ms	75 ms	75 ms
	Task 1.2	Node E (11000 J)	0 ms	0 ms	75 ms	75 ms
	Task 1.3	Node A (7040 J)	0 ms	0 ms	60 ms	60 ms
	Task 1.4	Node D(260 J)	0 ms	0 ms	25 ms	25 ms
Task2 (10589 J)	Task 2.1	Node D (6780 J)	0 ms	0 ms	50 ms	50 ms
	Task 2.2	Node B (3809 J)	0 ms	0 ms	40 ms	40 ms
Task3 (20689 J)	Task 3.1	Node B (3231 J)	0 ms	0 ms	38 ms	38 ms
	Task 3.2	Node C (11000 J)	0 ms	75 ms	140 ms	65 ms
	Task 3.3	Node E (6458 J)	0 ms	75 ms	150 ms	75 ms
Task 4 (4542 J)	Task 4	Node E(4542 J)	0 ms	75 ms	140 ms	65 ms
Task5 (13260 J)	Task 5.1	Node A (7040 J)	150 ms	0 ms	210 ms	60 ms
	Task 5.2	Node D (6220 J)	150 ms	0 ms	210 ms	60 ms
Task6 (12262 J)	Task 6.1	Node C (820 J)	150 ms	0 ms	165 ms	15 ms
	Task 6.2	Node B (7040 J)	150 ms	0 ms	210 ms	60 ms
	Task 6.3	Node C (4402 J)	150 ms	0 ms	190 ms	40 ms
Task7 (10589 J)	Task 7.1	Node C (6598 J)	150 ms	0 ms	205 ms	55 ms
	Task 7.2	Node E (3991 J)	150 ms	0 ms	185 ms	35 ms
Task8 (12262 J)	Task 8.1	Node E (7009 J)	150 ms	0 ms	210 ms	60 ms
	Task 8.2	Node A (7040 J)	150 ms	60 ms	270 ms	60 ms
	Task 8.3	Node D (6583 J)	150 ms	50 ms	255 ms	55 ms
Task9 (17450 J)	Task 9.1	Node D (457 J)	200 ms	0 ms	208 ms	8 ms
	Task 9.2	Node B (7040 J)	200 ms	10 ms	270 ms	60 ms
	Task 9.3	Node C (9953 J)	200 ms	5 ms	280 ms	75 ms
Task10 (18301 J)	Task 10.1	Node C (1047 J)	200 ms	5 ms	225 ms	20 ms
	Task 10.2	Node E (11000 J)	200 ms	10 ms	285 ms	75 ms
	Task 10.3	Node A (6254 J)	200 ms	70 ms	340 ms	70 ms

7.5. Comparative analysis of EDITORS with existing works

We evaluate the effectiveness of EDITORS against five state-of-the-art schemes: DDLO [66], DROO [41], DMRO [67], DRL (without TL, and SDN) [43] and DRL based offloading with TL [31]. The task offloading approaches used in the field currently are

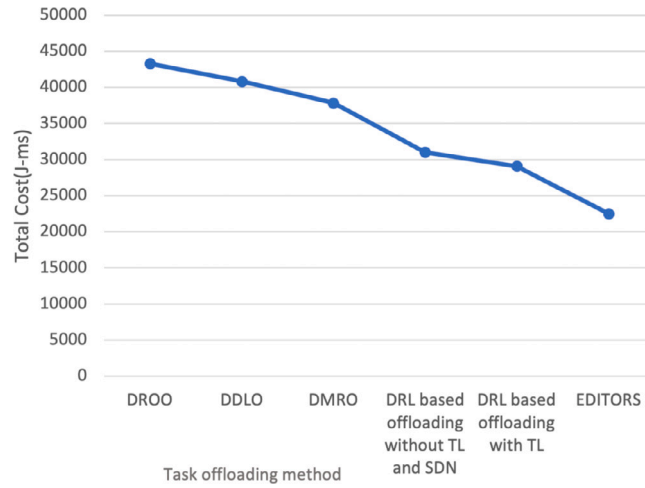


Fig. 10. Comparison of EDITORS method with existing works in terms of total cost.

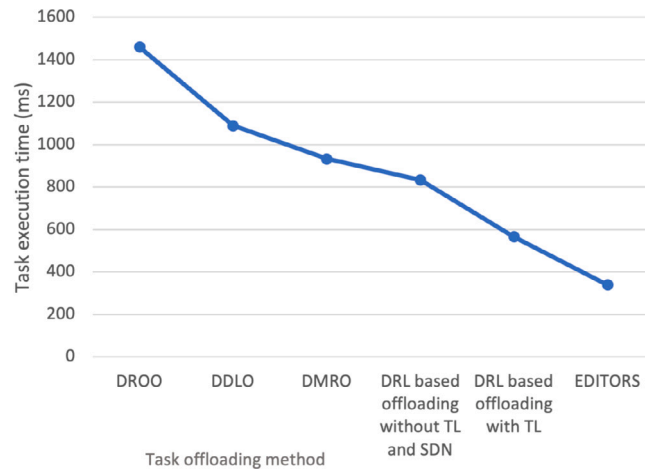


Fig. 11. Comparison of EDITORS method with existing works in terms of task execution time.

included in these latest works. We evaluated each method's performance in terms of task execution time (measured in milliseconds) and overall cost (measured in J-ms) in this comparison analysis. Our findings support our assertions that EDITORS is more suitable for partial task offloading and dynamic assignment of complex tasks among edge nodes at the lowest total cost (using Eqs. (1) and (2)) in terms of energy consumption, time delay, and trust score. EDITORS is particularly good when it comes to overall cost; at 22,500 J-ms, it is far lower than the nearest alternative. This impressive decrease in overall cost highlights our proposed algorithm's energy efficiency, which is an important factor in task offloading. Moreover, EDITORS outperforms all other approaches in comparison with an unprecedentedly low task execution time of just 340 ms. During the experiments, we initially allowed four edge nodes to start executing the tasks. Once the algorithm had reached a stable and optimised allocation of tasks, we introduced a new edge node into the system and incorporated it into the ongoing task allocation process. Figs. 10 and 11 depict the comparison of EDITORS with existing works in terms of total cost and task execution time.

Our EDITORS method results in a significantly shorter user-experienced training time, with a reduction of training time by 60%–70% compared to existing offloading methods because of the TL module in the DRTL framework. This efficient training time translates to substantial energy savings in practical implementations.

Although EDITORS exhibit many benefits, it is imperative to recognise that a thorough examination must consider any possible drawbacks or constraints. For example, there could be some overhead in terms of computational complexity and initial setup when implementing the TL module. Furthermore, the efficiency of any offloading algorithm could differ depending on the particulars of the network, device capabilities, and application needs.

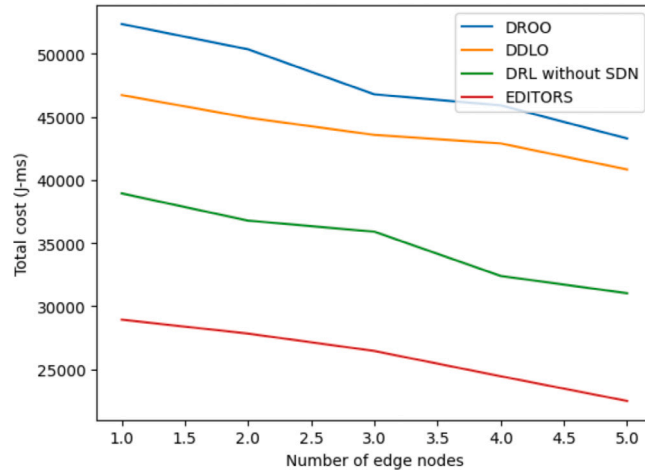


Fig. 12. Evaluation of current approaches using total cost as a benchmark.

7.6. Impact of number of edge nodes on total cost

Fig. 12 displays the comparative analysis of the overall cost associated with the proposed method in contrast to the costs of existing approaches. The cost is calculated using Eq. (1). The overall cost for offloading decreases in terms of energy consumption and latency as the number of edge nodes rises. In comparison to existing methods, EDITORS achieves low cost. This is due to the ability of DRTL with SDN to dynamically divide tasks into smaller tasks depending on energy usage and trust score, which in turn assigns task to various edge nodes based on their capabilities, improving the assignment of tasks, parallel computation, and resource optimisation. This may have a favourable effect on the system's overall cost effectiveness.

7.7. Impact of number of edge nodes on latency

As illustrated in Eqs. (2) and (3), EDITORS is designed to minimise network latency for available nodes during task execution. We computed the total time required for edge nodes to finish all pending task requests. Fig. 13 presents the gathered latency values observed during task offloading, comparing EDITORS with other methods. Consequently, the proposed approach consistently delivered minimal latency as the number of edge nodes increased. This is because the proposed solution uses SDN and TL. SDN enables the edge nodes to divide the tasks into subtasks and execute them in parallel across multiple edge nodes, and TL reduces the possibility of having to retrain the offloading models for newly connected devices.

7.8. Real system implementation and overhead analysis

Real-world scenarios highlight a number of potential overheads and challenges associated with applying the “EDITORS” approach to MEC. Reliable task offloading depends on controlling resource availability, including variances and contention among devices. System performance may be impacted by hardware limitations on real-world nodes and IoT devices, such as processor power and memory limits. There are overheads associated with safeguards, trust prediction, and privacy protections when it comes to security, trust assessment, and protection against attacks and breaches. Large amounts of computer power and training time may be needed when using DRTL and other TL approaches. Maintaining performance while ensuring system scalability is crucial when dealing with an increasing number of edge nodes and devices. It requires more validation and adjustment to translate simulation results into real-world performance. In order to accomplish the stated goals of low-latency task completion, energy efficiency, and trust management in mobile edge computing systems, these overheads must be addressed. It should be emphasised that our research just briefly discusses vehicle mobility, and that the system's evaluation does not take this into account, which can be seen as a limitation of the proposed system. More research is still needed to fully understand the importance of mobility, especially when vehicles are inside the user plane.

8. Conclusion

This study demonstrated the potential of creating the most appropriate task offloading systems utilising DRL and TRL for SDN-enabled IoT networks. We first suggested a SDN edge computing architecture that would allow the controller to break down tasks into subtasks and allocate them to the most appropriate edge nodes for execution. After that, a model is created for predicting the trust score of edge nodes based on energy consumption, latency, and memory usage utilising a feedback-based trust management method using LSTM. Then, we developed the EDITORS- DRTL offloading technique to achieve the most appropriate task offloading.

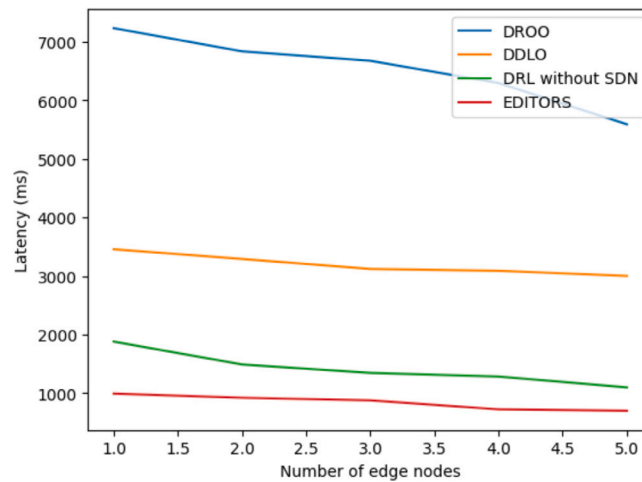


Fig. 13. Evaluation of current approaches using latency as a benchmark.

Ultimately, we were able to acquire the most appropriate edge node assignment schemes based on the energy usage and trust score of edge nodes using DRL. EDITORS addresses some of the key issues with task offloading in MEC, such as high training costs, reliability and limited adaptability of newly joined devices. DRTL requires less training time than the existing offloading method (60%–70%), which also results in substantial reductions in energy consumption. Additionally, our method guarantees minimal overall costs and latency in terms of energy consumption and task execution latency. Although this study has shown the potential of EDITORS offloading technique for SDN-enabled IoT networks, it is important to recognise certain limitations that might affect how applicable and generalisable our findings are. First, the quality and representativeness of the training data are prerequisites for our LSTM-based trust score prediction model. The performance of the model may be impacted by variations in real-world circumstances that were not fully captured in our training data. Furthermore, this study does not specifically address the security risks introduced by potential probing and eavesdropping attacks. Attacks like eavesdropping and probing could be possible due to the communications between the edge nodes. Moreover, the adaptability of our approach to different IoT network environments needs to be further examined, since the system's performance can change depending on certain network conditions or characteristics. Future research directions should investigate improved security mechanisms, model generalisation across various IoT deployments, and thorough scalability studies to overcome these limitations and guarantee that the EDITORS technique can be used effectively in a greater variety of circumstances. Future research can incorporate federated learning to address these security issues and strengthen and secure the offloading mechanism. Experiments using high mobility tools like UAV are intriguing for in-depth research.

CRediT authorship contribution statement

Thar Baker: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.
Zaher Al Aghbari: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.
Ahmed M. Khedr: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.
Naveed Ahmed: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.
Shini Girija: Data curation, Formal analysis, Investigation, Software, Validation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Han Hu, Dingguo Wu, Fuhui Zhou, Shi Jin, Rose Qingyang Hu, Dynamic task offloading in MEC-enabled IoT networks: A hybrid DDPG-d3qn approach, in: 2021 IEEE Global Communications Conference, GLOBECOM, IEEE, 2021, pp. 1–6.
- [2] Abednego Acheampong, Yiwen Zhang, Xiaolong Xu, Daniel Appiah Kumah, A review of the current task offloading algorithms, strategies and approach in edge computing systems, *Comput. Model. Eng. Sci.* 134 (1) (2023) 35–88.
- [3] Zhongyu Wang, Tiejun Lv, Zheng Chang, Computation offloading and resource allocation based on distributed deep learning and software defined mobile edge computing, *Comput. Netw.* 205 (2022) 108732.
- [4] Tianhao Zhang, Xiaojuan Zhu, Cai Wu, Reinforcement-learning-based software-defined edge task allocation algorithm, *Electronics* 12 (3) (2023) 773.
- [5] Mohammad Hassan Khoobkar, Mehdi Dehghan Takht Fooladi, Mohammad Hossein Rezvani, Mohammad Mehdi Gilanian Sadeghi, Joint optimization of delay and energy in partial offloading using dual-population replicator dynamics, *Expert Syst. Appl.* 216 (2023) 119417.
- [6] Firdose Saeik, Marios Avgeris, Dimitrios Spatharakis, Nina Santi, Dimitrios Dechouniotis, John Violos, Aris Leivadreas, Nikolaos Athanasopoulos, Nathalie Mitton, Symeon Papavassiliou, Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions, *Comput. Netw.* 195 (2021) 108177.
- [7] Anil Acharya, Yantian Hou, Ying Mao, Min Xian, Jiawei Yuan, Workload-aware task placement in edge-assisted human re-identification, in: 2019 16th Annual IEEE International Conference on Sensing, Communication, and Networking, SECON, IEEE, 2019, pp. 1–9.
- [8] Ganesh Ananthanarayanan, Paramvir Bahl, Peter Bodík, Krishna Chintalapudi, Matthai Philipose, Lenin Ravindranath, Sudipta Sinha, Real-time video analytics: The killer app for edge computing, *Computer* 50 (10) (2017) 58–67.
- [9] Yazhi Liu, Wendong Wang, Yuekun Ma, Zhigang Yang, Fuxing Yu, Distributed task offloading in heterogeneous vehicular crowd sensing, *Sensors* 16 (7) (2016) 1090.
- [10] Hank H. Harvey, Ying Mao, Yantian Hou, Bo Sheng, Edos: Edge assisted offloading system for mobile devices, in: 2017 26th International Conference on Computer Communication and Networks, ICCCN, IEEE, 2017, pp. 1–9.
- [11] Jung-yeon Baek, Georges Kaddoum, Sahil Garg, Kuljeet Kaur, Vivianne Gravel, Managing fog networks using reinforcement learning based load balancing algorithm, in: 2019 IEEE Wireless Communications and Networking Conference, WCNC, IEEE, 2019, pp. 1–7.
- [12] Bassem Sellami, Akram Hakiri, Sadok Ben Yahia, Pascal Berthou, Energy-aware task scheduling and offloading using deep reinforcement learning in SDN-enabled IoT network, *Comput. Netw.* 210 (2022) 108957.
- [13] Ducsun Lim, Inwhae Joe, A DRL-based task offloading scheme for server decision-making in multi-access edge computing, *Electronics* 12 (18) (2023) 3882.
- [14] Hyame Assem Alameddine, Sanaa Sharafeddine, Samir Sebbah, Sara Ayoubi, Chadi Assi, Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing, *IEEE J. Sel. Areas Commun.* 37 (3) (2019) 668–682.
- [15] Xinchun Lyu, Hui Tian, Cigdem Sengul, Ping Zhang, Multiuser joint task offloading and resource optimization in proximate clouds, *IEEE Trans. Veh. Technol.* 66 (4) (2016) 3435–3447.
- [16] Xiaolong Xu, Yuancheng Li, Tao Huang, Yuan Xue, Kai Peng, Lianyong Qi, Wanchun Dou, An energy-aware computation offloading method for smart edge computing in wireless metropolitan area networks, *J. Netw. Comput. Appl.* 133 (2019) 75–85.
- [17] Mohammad Nikravan, Mostafa Haghi Kashani, A review on trust management in fog/edge computing: Techniques, trends, and challenges, *J. Netw. Comput. Appl.* 204 (2022) 103402.
- [18] Tian Wang, Lei Qiu, Arun Kumar Sangaiah, Anfeng Liu, Md Zakirul Alam Bhuiyan, Ying Ma, Edge-computing-based trustworthy data collection model in the internet of things, *IEEE Internet Things J.* 7 (5) (2020) 4218–4227.
- [19] Klervie Toczé, Simin Nadim-Tehrani, A taxonomy for management and optimization of multiple resources in edge computing, *Wirel. Commun. Mob. Comput.* 2018 (2018).
- [20] Pawani Porambage, Jude Okwuibe, Madhusanka Liyanage, Mika Ylianttila, Tarik Taleb, Survey on multi-access edge computing for internet of things realization, *IEEE Commun. Surv. Tutor.* 20 (4) (2018) 2961–2991.
- [21] Zemin Liu, Na Zhou, Yan Wang, Jian-Tao Zhou, Haotian Zhang, Gang Xu, et al., An effective task offloading method for separable complex mobile terminal tasks, *Wirel. Commun. Mob. Comput.* 2022 (2022).
- [22] Hoa Tran-Dang, Dong-Seong Kim, FRATO: Fog resource based adaptive task offloading for delay-minimizing IoT service provisioning, *IEEE Trans. Parallel Distrib. Syst.* 32 (10) (2021) 2491–2508.
- [23] Kaile Xiao, Zhipeng Gao, Weisong Shi, Xuesong Qiu, Yang Yang, Lanlan Rui, EdgeABC: An architecture for task offloading and resource allocation in the internet of things, *Future Gener. Comput. Syst.* 107 (2020) 498–508.
- [24] Suhail Ahmad, Ajaz Hussain Mir, Scalability, consistency, reliability and security in SDN controllers: a survey of diverse SDN controllers, *J. Netw. Syst. Manage.* 29 (2021) 1–59.
- [25] Yustus Eko Oktian, SangGon Lee, HoonJae Lee, JunHuy Lam, Distributed SDN controller system: A survey on design choice, *Comput. Netw.* 121 (2017) 100–111.
- [26] Mithun Mukherjee, Rakesh Matam, Constandinos X. Mavromoustakis, Hao Jiang, George Mastorakis, Mian Guo, Intelligent edge computing: Security and privacy challenges, *IEEE Commun. Mag.* 58 (9) (2020) 26–31.
- [27] Karthik Kumar, Yamini Nimmagadda, Yung-Hsiang Lu, Establishing trust for computation offloading, in: 2009 Proceedings of 18th International Conference on Computer Communications and Networks, IEEE, 2009, pp. 1–6.
- [28] Huy T. Nguyen, Nguyen Cong Luong, Jun Zhao, Chau Yuen, Dusit Niyato, Resource allocation in mobility-aware federated learning networks: A deep reinforcement learning approach, in: 2020 IEEE 6th World Forum on Internet of Things, WF-IoT, IEEE, 2020, pp. 1–6.
- [29] Howard H. Yang, Zuozhu Liu, Tony Q.S. Quek, H. Vincent Poor, Scheduling policies for federated learning in wireless networks, *IEEE Trans. Commun.* 68 (1) (2019) 317–333.
- [30] Gaith Rjoub, Omar Abdel Wahab, Jamal Bentahar, Ahmed Bataineh, A trust and energy-aware double deep reinforcement learning scheduling strategy for federated learning on IoT devices, in: Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings 18, Springer, 2020, pp. 319–333.
- [31] Kefan Shuai, Yiming Miao, Kai Hwang, Zhengdao Li, Transfer reinforcement learning for adaptive task offloading over distributed edge clouds, *IEEE Trans. Cloud Comput.* (2022).
- [32] Jie Li, Zhiping Yang, Xingwei Wang, Yichao Xia, Shijian Ni, Task offloading mechanism based on federated reinforcement learning in mobile edge computing, *Digit. Commun. Netw.* 9 (2) (2023) 492–504.
- [33] Sudip Misra, Niloy Saha, Detour: Dynamic task offloading in software-defined fog for IoT applications, *IEEE J. Sel. Areas Commun.* 37 (5) (2019) 1159–1166.
- [34] Nahida Kiran, Chunyu Pan, Sihua Wang, Changchuan Yin, Joint resource allocation and computation offloading in mobile edge computing for SDN based wireless networks, *J. Commun. Netw.* 22 (1) (2019) 1–11.
- [35] Gianmarco Lia, Marica Amadeo, Giuseppe Ruggeri, Claudia Campolo, Antonella Molinaro, Valeria Loscri, In-network placement of delay-constrained computing tasks in a softwareized intelligent edge, *Comput. Netw.* 219 (2022) 109432.
- [36] Tabassum N. Mujaawar, Lokesh B. Bhajantri, Behavior and feedback based trust computation in cloud environment, *J. King Saud Univ.-Comput. Inf. Sci.* 34 (8) (2022) 4956–4967.

- [37] Chuanyi Liu, Xiaoyong Li, Mingliang Sun, Yali Gao, Jie Yuan, Shaoming Duan, Bi-TCCS: Trustworthy cloud collaboration service scheme based on bilateral social feedback, *IEEE Trans. Cloud Comput.* 10 (2) (2020) 1021–1037.
- [38] Liang Huang, Xu Feng, Anqi Feng, Yupin Huang, Li Ping Qian, Distributed deep learning-based offloading for mobile edge computing networks, *Mobile Netw. Appl.* (2018) 1–8.
- [39] Bin Cao, Long Zhang, Yun Li, Daquan Feng, Wei Cao, Intelligent offloading in multi-access edge computing: A state-of-the-art review and framework, *IEEE Commun. Mag.* 57 (3) (2019) 56–62.
- [40] Wei Ying, Yu Zhang, Junzhou Huang, Qiang Yang, Transfer learning via learning to transfer, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 5085–5094.
- [41] Liang Huang, Suzhi Bi, Ying-Jun Angela Zhang, Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks, *IEEE Trans. Mob. Comput.* 19 (11) (2019) 2581–2593.
- [42] Ibrahim A. Elgendy, Wei-Zhe Zhang, Hui He, Brij B. Gupta, Ahmed A. Abd El-Latif, Joint computation offloading and task caching for multi-user and multi-task MEC systems: reinforcement learning-based algorithms, *Wirel. Netw.* 27 (3) (2021) 2023–2038.
- [43] Nguyen Van Tam, Nguyen Quang Hieu, Nguyen Thi Thanh Van, Nguyen Cong Luong, Dusit Niyato, Dong In Kim, Adaptive task offloading in coded edge computing: A deep reinforcement learning approach, *IEEE Commun. Lett.* 25 (12) (2021) 3878–3882.
- [44] Haibo Zhang, Zixin Wang, Kaijian Liu, V2X offloading and resource allocation in SDN-assisted MEC-based vehicular networks, *China Commun.* 17 (5) (2020) 266–283.
- [45] Liang Zhao, Zijia Zhao, Enchao Zhang, Ammar Hawbani, Ahmed Al-Dubai, Zhiyuan Tan, Amir Hussain, A digital twin-assisted intelligent partial offloading approach for vehicular edge computing, *IEEE J. Sel. Areas Commun.* (2023).
- [46] Liang Zhao, Kaiqi Yang, Zhiyuan Tan, Xianwei Li, Suraj Sharma, Zhi Liu, A novel cost optimization strategy for SDN-enabled UAV-assisted vehicular computation offloading, *IEEE Trans. Intell. Transp. Syst.* 22 (6) (2020) 3664–3674.
- [47] Jin Wang, Jia Hu, Geyong Min, Albert Y. Zomaya, Nektarios Georgalas, Fast adaptive task offloading in edge computing based on meta reinforcement learning, *IEEE Trans. Parallel Distrib. Syst.* 32 (1) (2020) 242–253.
- [48] AbdelRahman Abdou, Paul C. Van Oorschot, Tao Wan, Comparative analysis of control plane security of SDN and conventional networks, *IEEE Commun. Surv. Tutor.* 20 (4) (2018) 3542–3559.
- [49] Mingjie Feng, Marwan Krunz, Wenhan Zhang, Joint task partitioning and user association for latency minimization in mobile edge computing networks, *IEEE Trans. Veh. Technol.* 70 (8) (2021) 8108–8121.
- [50] Yun Zhao, Sheng Zhou, Tianchu Zhao, Zhisheng Niu, Energy-efficient task offloading for multiuser mobile cloud computing, in: *2015 IEEE/CIC International Conference on Communications in China, ICCIC, IEEE*, 2015, pp. 1–5.
- [51] Jindou Xie, Yunjian Jia, Zhengchuan Chen, Zhaojun Nan, Liang Liang, D2D computation offloading optimization for precedence-constrained tasks in information-centric IoT, *IEEE Access* 7 (2019) 94888–94898.
- [52] Pavel Mach, Zdenek Becvar, Mobile edge computing: A survey on architecture and computation offloading, *IEEE Commun. Surv. Tutor.* 19 (3) (2017) 1628–1656.
- [53] Farzad Samie, Vasileios Tsoutsouras, Lars Bauer, Sotirios Xydis, Dimitrios Soudris, Jörg Henkel, Computation offloading and resource allocation for low-power IoT edge devices, in: *2016 IEEE 3rd World Forum on Internet of Things, WF-IoT, IEEE*, 2016, pp. 7–12.
- [54] Congfeng Jiang, Tiantian Fan, Honghao Gao, Weisong Shi, Liangkai Liu, Christophe Cérin, Jian Wan, Energy aware edge computing: A survey, *Comput. Commun.* 151 (2020) 556–580.
- [55] SGOPAL Patro, Kishore Kumar Sahu, Normalization: A preprocessing stage, 2015, *arXiv preprint arXiv:1503.06462*.
- [56] Jie Wu, Jiasen Sun, Liang Liang, Yingchun Zha, Determination of weights for ultimate cross efficiency using Shannon entropy, *Expert Syst. Appl.* 38 (5) (2011) 5162–5165.
- [57] Maryam Khani, Yan Wang, Mehmet A. Orgun, Feng Zhu, Context-aware trustworthy service evaluation in social internet of things, in: *Service-Oriented Computing: 16th International Conference, ICSOC 2018, Hangzhou, China, November 12–15, 2018, Proceedings 16*, Springer, 2018, pp. 129–145.
- [58] Gaith Rjoub, Jamal Bentahar, Omar Abdel Wahab, Ahmed Saleh Bataineh, Deep and reinforcement learning for automated task scheduling in large-scale cloud computing systems, *Concurr. Comput.: Pract. Exper.* 33 (23) (2021) e5919.
- [59] Jianqing Fan, Zhaoran Wang, Yuchen Xie, Zhuoran Yang, A theoretical analysis of deep Q-learning, in: *Learning for Dynamics and Control*, PMLR, 2020, pp. 486–489.
- [60] Taha Alfakih, Mohammad Mehdi Hassan, Abdu Gumaie, Claudio Savaglio, Giancarlo Fortino, Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA, *IEEE Access* 8 (2020) 54074–54084.
- [61] Julio C.S. Dos Anjos, João L.G. Gross, Kassiano J. Matteussi, Gabriel V. González, Valderi R.Q. Leithardt, Claudio F.R. Geyer, An algorithm to minimize energy consumption and elapsed time for IoT workloads in a hybrid architecture, *Sensors* 21 (9) (2021) 2914.
- [62] Yubin Deng, Ping Luo, Chen Change Loy, Xiaou Tang, Pedestrian attribute recognition at far distance, in: *Proceedings of the 22nd ACM International Conference on Multimedia*, 2014, pp. 789–792.
- [63] Yong Yu, Xiaosheng Si, Changhua Hu, Jianxun Zhang, A review of recurrent neural networks: LSTM cells and network architectures, *Neural Comput.* 31 (7) (2019) 1235–1270.
- [64] Shanu Bhardwaj, Surya Narayan Panda, Performance evaluation using RYU SDN controller in software-defined networking environment, *Wirel. Pers. Commun.* 122 (1) (2022) 701–723.
- [65] Seungwoon Lee, Jehad Ali, Byeong-hee Roh, Performance comparison of software defined networking simulators for tactical network: Mininet vs. OPNET, in: *2019 International Conference on Computing, Networking and Communications, ICNC, IEEE*, 2019, pp. 197–202.
- [66] Peizhi Yan, Salimur Choudhury, Deep Q-learning enabled joint optimization of mobile edge computing multi-level task offloading, *Comput. Commun.* 180 (2021) 271–283.
- [67] Guanjin Qu, Huaming Wu, Ruidong Li, Pengfei Jiao, DMRO: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing, *IEEE Trans. Netw. Serv. Manag.* 18 (3) (2021) 3448–3459.